SISTEMAS DIGITAIS II - 2026/27

TECNOLOGIA SETÚBAL
ESCOLA SUPERIOR POLITÉCNICO SETÚBAL

Laboratório Nº	1
Data	09/10/2026
Docente	José Sousa
Curso	LEEC
Turma	PL1

Alunos		
	202300553	Rodrigo Silva Costa
	2025184378	Diego Alejandro Castro Laya

1.º Trabalho de Laboratório

Gestão do display da placa Basys3

(Tutorial Digital e Vivado)

Índice

1 Introdução....2
2 Multiplexer 4x1....3
2.1 Desenho do circuito no Digital....3
2.2 Simulação no Digital....3
2.3 Exportação para Verilog....4
2.4 Projeto no Vivado e ficheiro de restrições....4
2.5 Testbench e simulação....4
2.6 Geração do bitstream e ensaio na Basys3....6
3 Descodificador 2x4 com enables individuais....8
4 Multiplexer 16x4....11
5 Transcodificador BCD para display de 7 segmentos....14
5.1 Ensaio do transcodificador na placa....15
6 Registo de 4 bits com clear e enable....18
7 Divisor de clock e contador de módulo 4....19
7.1 Divisor de clock....19
7.2 Contador de módulo 4....19
8 Controlador dos displays da placa Basys3....21
8.1 Circuito de teste na placa....24
9 Análise crítica do código de fontes externas....28
9.1 Código gerado pelo Digital....28
9.2 Transcodificador do Moodle....28
9.3 Divisores de clock do Moodle....28
9.4 Contadores do Moodle....28
9.5 Primitiva FDCE....28
9.6 IA generativa....29
10 Conclusão....30
Anexo A Testbenches....31

1 Introdução

Este relatório descreve o 1.º Trabalho de Laboratório de Sistemas Digitais II: o controlo do *display* de 7 segmentos e 4 dígitos da placa Basys3, percorrendo o fluxo completo de projeto no Vivado (versão 2026.1, licença WebPACK) e no Digital (versão v0.31).

Os módulos mais simples foram desenhados e simulados no Digital e exportados para Verilog; os restantes foram escritos diretamente no Vivado. Para cada módulo apresentam-se o código, as restrições, o *testbench* e a simulação comentada; as listagens dos *testbenches*, exceto a primeira (secção 2.5), estão no Anexo A.

2 Multiplexer 4x1

2.1 Desenho do circuito no Digital

O *multiplexer* 4x1 foi desenhado no Digital com quatro portas AND de 3 entradas e uma OR de 4 entradas (Figura 1): para cada combinação da seleção, só uma das AND deixa passar a respetiva entrada:

$$DataOut = Sel1' \cdot Sel0' \cdot DataIn0 + Sel1' \cdot Sel0 \cdot DataIn1 + Sel1 \cdot Sel0' \cdot DataIn2 + Sel1 \cdot Sel0 \cdot DataIn3$$

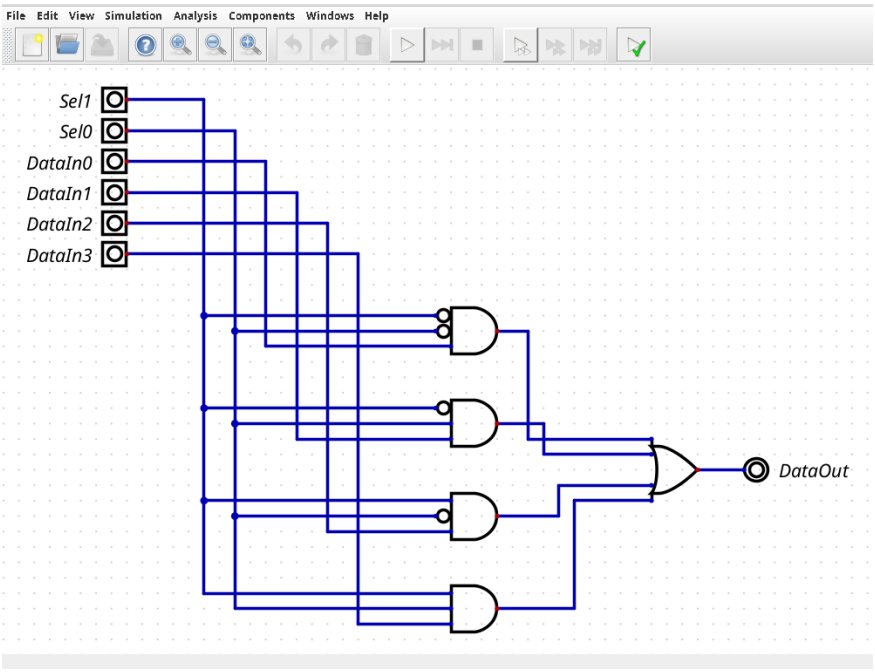


Figura 1: Circuito final do multiplexer 4x1 desenhado no Digital.

2.2 Simulação no Digital

A Figura 2 mostra um dos ensaios no simulador do Digital: com Sel1 Sel0 = 10 a saída segue DataIn2 (= 1), apesar de DataIn0 também estar a 1. O ensaio foi repetido para as quatro seleções.

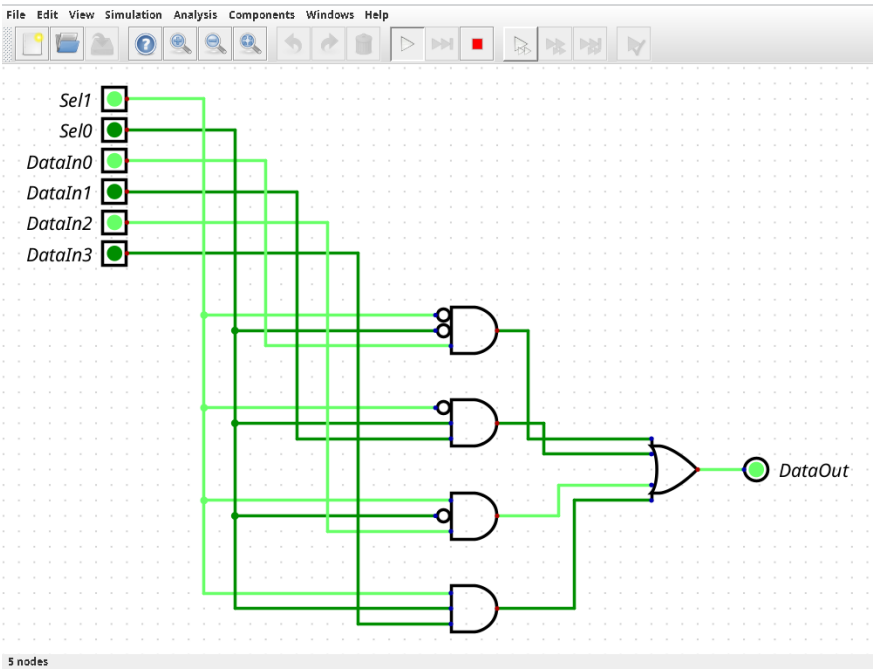


Figura 2: Simulação do multiplexer 4x1 no Digital (Sel1 = 1, Sel0 = 0, DataIn0 = DataIn2 = 1, DataOut = 1).

2.3 Exportação para Verilog

O circuito foi exportado com File > Export > Export to Verilog (Listagem 1).

Listagem 1: Código Verilog do multiplexer 4x1 gerado pelo Digital (`multiplexer4x1/multiplexer4x1.v`)

```
/*
 * Generated by Digital. Don't modify this file!
 * Any changes will be lost if this file is regenerated.
 */

module multiplexer4x1 (
  input Sel1,
  input Sel0,
  input DataIn0,
  input DataIn1,
  input DataIn2,
  input DataIn3,
  output DataOut
);
  assign DataOut = ((~ Sel1 & ~ Sel0 & DataIn0) | (~ Sel1 & Sel0 & DataIn1) | (Sel1 & ~ Sel0 & DataIn2) | (Sel1 & Sel0 & DataIn3));
endmodule
```

2.4 Projeto no Vivado e ficheiro de restrições

No Vivado criou-se o projeto `multiplexer4x1` para a FPGA da Basys3 (`xc7a35ticpg236-1L`). Nas restrições (Listagem 2) as entradas de dados ficaram nos interruptores SW0 a SW3, em vez das teclas do enunciado, para o estado de todas as entradas ficar visível; a seleção ficou em SW5 e SW4 e a saída no LED LD0.

Listagem 2: Ficheiro de restrições do multiplexer 4x1 (`multiplexer4x1/multiplexer4x1.xdc`)

```
# Variáveis de seleção do multiplexer 4x1 (interruptores SW5 e SW4)
set_property PACKAGE_PIN V15 [get_ports Sel1]
    set_property IOSTANDARD LVCMOS33 [get_ports Sel1]
set_property PACKAGE_PIN W15 [get_ports Sel0]
    set_property IOSTANDARD LVCMOS33 [get_ports Sel0]

# Variáveis de entrada do multiplexer 4x1 (interruptores SW0 a SW3)
set_property PACKAGE_PIN V17 [get_ports DataIn0]
    set_property IOSTANDARD LVCMOS33 [get_ports DataIn0]
set_property PACKAGE_PIN V16 [get_ports DataIn1]
    set_property IOSTANDARD LVCMOS33 [get_ports DataIn1]
set_property PACKAGE_PIN W16 [get_ports DataIn2]
    set_property IOSTANDARD LVCMOS33 [get_ports DataIn2]
set_property PACKAGE_PIN W17 [get_ports DataIn3]
    set_property IOSTANDARD LVCMOS33 [get_ports DataIn3]

# Variável de saída do multiplexer 4x1 (LED LD0)
set_property PACKAGE_PIN U16 [get_ports DataOut]
    set_property IOSTANDARD LVCMOS33 [get_ports DataOut]

# Definir a tensão correta da placa Basys3
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

2.5 Testbench e simulação

O *testbench* (Listagem 3) segue a estrutura do enunciado (entradas `reg`, saída `wire`, instância UUT e ciclo `for` com `case`, de 50 em 50 ns). Para cada seleção coloca a 1 primeiro **só** a entrada selecionada (Z =

1) e depois **todas as outras** (Z = 0), o que mostra que só a entrada selecionada influencia a saída (Tabela 1). Um bloco `always` compara `Z` com `D[S]` e escreve o resultado na consola.

Tabela 1: Sequência aplicada no testbench do multiplexer 4x1 e valores esperados de Z.

Instante (ns)	S	D (DataIn3..0)	Z esperado	Observação
0	00	0000	0	Estado inicial
250	00	0001	1	Só a entrada 0 a 1
300	00	1110	0	Todas a 1 menos a 0
350	01	1110	1	Seleciona a entrada 1
400	01	0010	1	Só a entrada 1 a 1
450	01	1101	0	Todas a 1 menos a 1
500	10	1101	1	Seleciona a entrada 2
550	10	0100	1	Só a entrada 2 a 1
600	10	1011	0	Todas a 1 menos a 2
650	11	1011	1	Seleciona a entrada 3
700	11	1000	1	Só a entrada 3 a 1
750	11	0111	0	Todas a 1 menos a 3
800	11	1111	1	Todas a 1
850 / 900 / 950	10 / 01 / 00	1111	1	Seleção percorre 2, 1 e 0
1000	00	0000	0	Todas a 0

Listagem 3: Testbench do multiplexer 4x1 (`multiplexer4x1/multiplexer4x1_teste1.v`)

```
1  `timescale 1ns / 1ps
2
3  // Testbench do multiplexer 4x1 (sequência personalizada)
4  module multiplexer4x1_teste1();
5
6  //   Declaração das entradas
7       reg [1:0] S;
8       reg [3:0] D;
9
10 //   Declaração da saída
11      wire Z;
12
13 //   Instanciação da UUT e ligação das entradas e saídas
14      multiplexer4x1 UUT (
15          .Sel1(S[1]),
16          .Sel0(S[0]),
17          .DataIn0(D[0]),
18          .DataIn1(D[1]),
19          .DataIn2(D[2]),
20          .DataIn3(D[3]),
21          .DataOut(Z)
22      );
23
24 //   Declaração de vars auxiliares
25      integer n;            // Contador para o ciclo for
26      integer erros = 0;    // Número de saídas diferentes do esperado
27
28 //   Verificação automática: a saída deve ser sempre igual à entrada selecionada
29      always @(S or D) begin
30          #1;
31          if (Z !== D[S]) begin
32              $display("ERRO t=%0t: S=%b D=%b Z=%b (esperado %b)", $time, S, D, Z, D[S]);
33              erros = erros + 1;
34          end
```

```
35      end
36
37  //  Definição do valor das entradas ao longo do tempo
38  //  Para cada seleção: liga só a entrada selecionada (Z=1), depois liga todas
39  //  as outras exceto a selecionada (Z=0), mostrando que só a selecionada passa.
40      initial begin
41          S = 2'b00;        // Valor inicial das vars de seleção
42          D = 4'b0000;      // Valor inicial das vars de entrada (Z=0)
43
44          #200;             // Aguardar 200ns para simular o tempo
45                            // das configurações iniciais da FPGA
46
47          for (n=0; n<16; n=n+1)
48          begin
49              #50;                        // Passam 50ns a cada ciclo de n
50              case (n)
51                   0: D = 4'b0001;        // Entrada 0 a 1, selecionada a 0 (Z=1)
52                   1: D = 4'b1110;        // Todas a 1 menos a 0 (Z=0)
53                   2: S = 2'b01;          // Seleciona a entrada 1 (Z=1)
54                   3: D = 4'b0010;        // Só a entrada 1 a 1 (Z=1)
55                   4: D = 4'b1101;        // Todas a 1 menos a 1 (Z=0)
56                   5: S = 2'b10;          // Seleciona a entrada 2 (Z=1)
57                   6: D = 4'b0100;        // Só a entrada 2 a 1 (Z=1)
58                   7: D = 4'b1011;        // Todas a 1 menos a 2 (Z=0)
59                   8: S = 2'b11;          // Seleciona a entrada 3 (Z=1)
60                   9: D = 4'b1000;        // Só a entrada 3 a 1 (Z=1)
61                  10: D = 4'b0111;        // Todas a 1 menos a 3 (Z=0)
62                  11: D = 4'b1111;        // Todas a 1 (Z=1)
63                  12: S = 2'b10;          // Seleciona a entrada 2 (Z=1)
64                  13: S = 2'b01;          // Seleciona a entrada 1 (Z=1)
65                  14: S = 2'b00;          // Seleciona a entrada 0 (Z=1)
66                  15: D = 4'b0000;        // Todas a 0 (Z=0)
67              endcase
68          end
69          #50;
70          if (erros == 0) $display("Simulacao OK: multiplexer4x1 sem erros");
71          else            $display("Simulacao FALHOU: %0d erros", erros);
72          $finish;
73      end
74  endmodule
```

Na simulação (Figura 3) z coincide em todos os intervalos com a Tabela 1; a consola indicou `Simulacao OK: multiplexer4x1 sem erros`.

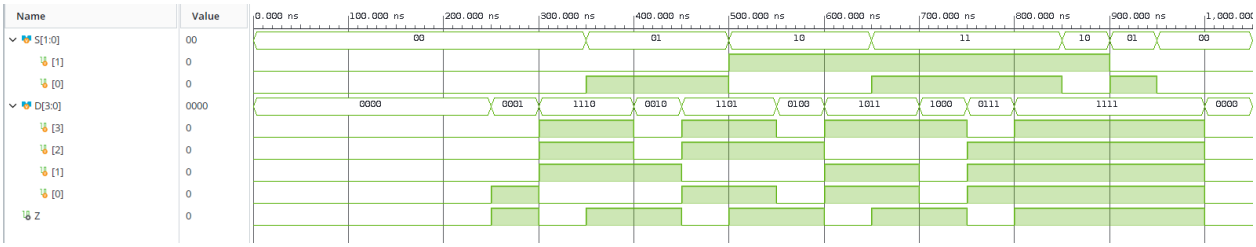


Figura 3: Simulação comportamental do multiplexer 4x1 no Vivado.

2.6 Geração do bitstream e ensaio na Basys3

A síntese, a implementação e a geração do *bitstream* decorreram sem erros (1 LUT: a função de 6 variáveis cabe numa LUT da Artix-7) e o ficheiro foi carregado com o Adept. Na placa (Figura 4), SW5 SW4 = 01 seleciona SW1, que está a 1, e o LED LD0 acende.

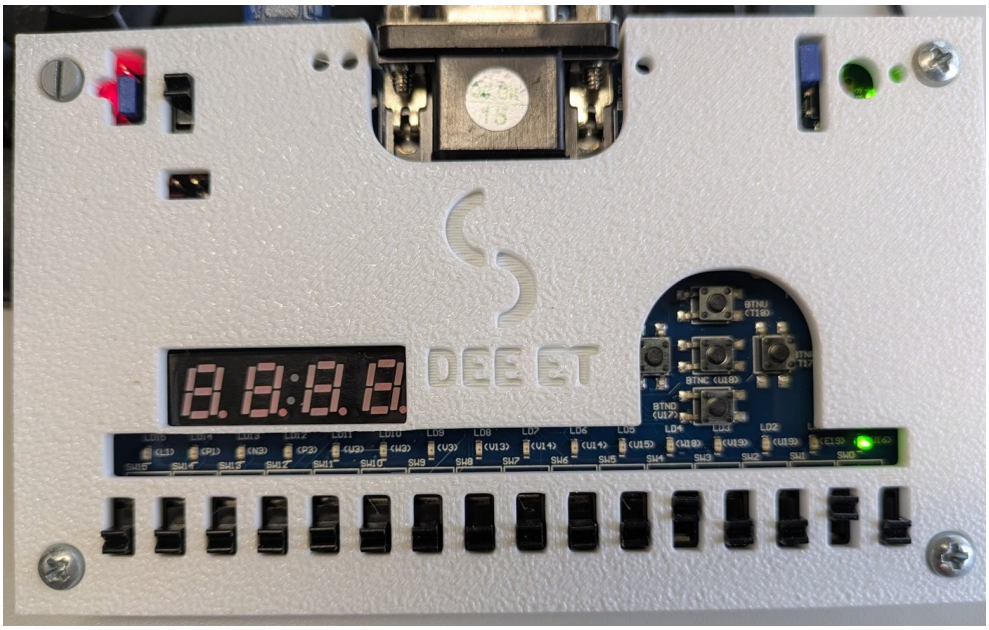


Figura 4: Ensaio do multiplexer 4x1 na placa Basys3: seleção 01 e SW1 = 1, LD0 aceso.

3 Descodificador 2x4 com enables individuais

Desenhou-se um descodificador de 2 *bits* com um *enable* por saída (Fig. 26 do enunciado): a saída selecionada por Bit1 Bit0 só fica a 1 se o seu *enable* estiver a 1, e os outros *enables* não têm efeito. No controlador (secção 8) escolhe o *display* ativo e os *enables* ligam ou desligam cada *display*.

$$Out0 = Bit1' \cdot Bit0' \cdot En0 \quad Out1 = Bit1' \cdot Bit0 \cdot En1 \quad Out2 = Bit1 \cdot Bit0' \cdot En2 \quad Out3 = Bit1 \cdot Bit0 \cdot En3$$

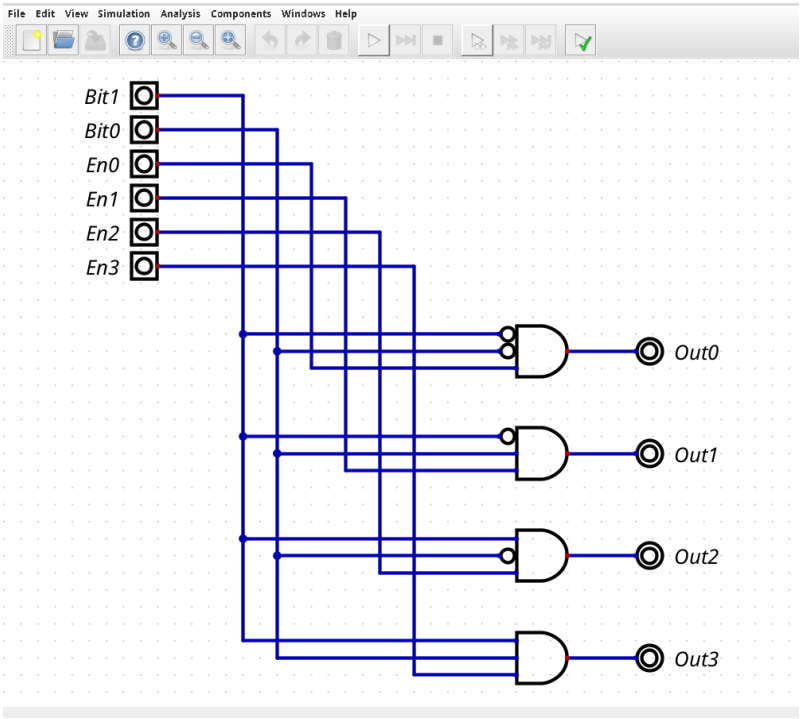


Figura 5: Esquema do descodificador 2x4 com enables individuais desenhado no Digital.

Na simulação no Digital (Figura 6), com Bit1 Bit0 = 10 e En1 = En2 = 1, só Out2 fica a 1 (Tabela 2).

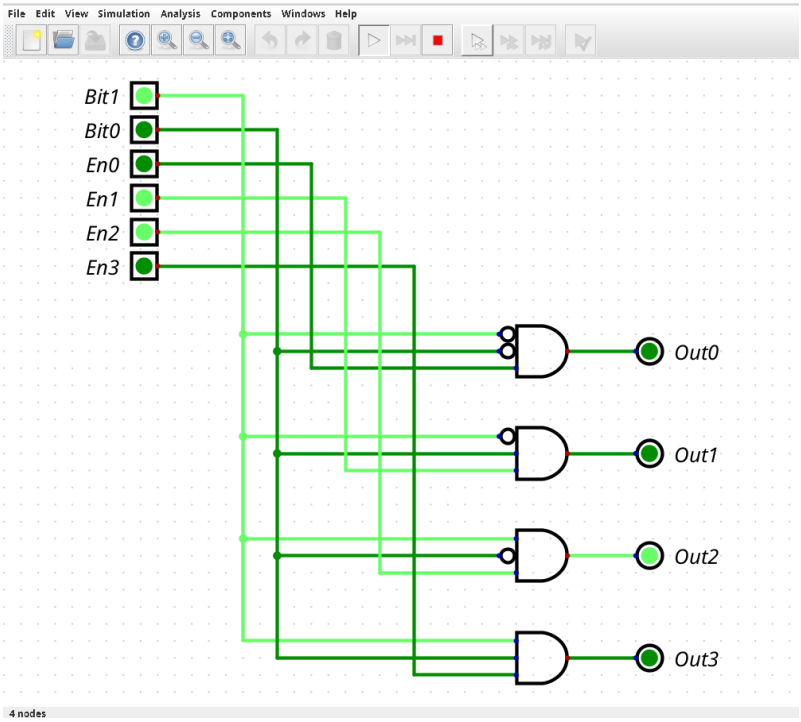


Figura 6: Simulação do descodificador 2x4 no Digital (Bit1 = 1, Bit0 = 0, En1 = En2 = 1): só Out2 fica a 1.

Tabela 2: Funcionamento do descodificador 2x4 com enables individuais (X = indiferente).

Bit1	Bit0	En3 En2 En1 En0	Out3 Out2 Out1 Out0
0	0	X X X 1	0 0 0 1
0	1	X X 1 X	0 0 1 0
1	0	X 1 X X	0 1 0 0
1	1	1 X X X	1 0 0 0
X	X	Eni = 0 (i = código selecionado)	0 0 0 0

O código exportado (Listagem 4) tem uma atribuição contínua por saída. Nas restrições (Listagem 5) a seleção ficou em SW15 e SW14, os *enables* em SW0 a SW3 e as saídas em LD0 a LD3, cada *enable* por baixo do LED que controla.

Listagem 4: Código Verilog do descodificador 2x4 gerado pelo Digital (`descodificador2x4/descodificador2x4.v`)

```
/*
 * Generated by Digital. Don't modify this file!
 * Any changes will be lost if this file is regenerated.
 */

module descodificador2x4 (
  input Bit1,
  input Bit0,
  input En0,
  input En1,
  input En2,
  input En3,
  output Out0,
  output Out1,
  output Out2,
  output Out3
);
  assign Out0 = (~ Bit1 & ~ Bit0 & En0);
  assign Out1 = (~ Bit1 & Bit0 & En1);
  assign Out2 = (Bit1 & ~ Bit0 & En2);
  assign Out3 = (Bit1 & Bit0 & En3);
endmodule
```

Listagem 5: Ficheiro de restrições do descodificador 2x4 (`descodificador2x4/descodificador2x4.xdc`)

```
# Variáveis de entrada do descodificador 2x4 (interruptores SW15 e SW14)
set_property PACKAGE_PIN R2 [get_ports Bit1]
    set_property IOSTANDARD LVCMOS33 [get_ports Bit1]
set_property PACKAGE_PIN T1 [get_ports Bit0]
    set_property IOSTANDARD LVCMOS33 [get_ports Bit0]

# Enables individuais das saídas (interruptores SW0 a SW3)
set_property PACKAGE_PIN V17 [get_ports En0]
    set_property IOSTANDARD LVCMOS33 [get_ports En0]
set_property PACKAGE_PIN V16 [get_ports En1]
    set_property IOSTANDARD LVCMOS33 [get_ports En1]
set_property PACKAGE_PIN W16 [get_ports En2]
    set_property IOSTANDARD LVCMOS33 [get_ports En2]
set_property PACKAGE_PIN W17 [get_ports En3]
    set_property IOSTANDARD LVCMOS33 [get_ports En3]

# Variáveis de saída do descodificador 2x4 (LEDs LD0 a LD3)
set_property PACKAGE_PIN U16 [get_ports Out0]
    set_property IOSTANDARD LVCMOS33 [get_ports Out0]
```

```
set_property PACKAGE_PIN E19 [get_ports Out1]
    set_property IOSTANDARD LVCMOS33 [get_ports Out1]
set_property PACKAGE_PIN U19 [get_ports Out2]
    set_property IOSTANDARD LVCMOS33 [get_ports Out2]
set_property PACKAGE_PIN V19 [get_ports Out3]
    set_property IOSTANDARD LVCMOS33 [get_ports Out3]

# Definir a tensão correta da placa Basys3
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

O *testbench* (Listagem 17, no Anexo A) percorre os quatro códigos com todos os *enables* a 1, desliga depois os *enables* um a um e confirma que o *enable* de uma saída não selecionada não acende nada; a saída é comparada com `(4'b0001 << B) & E`. Na Figura 7 a saída ativa percorre O0 a O3 entre 250 e 400 ns e passa a 0000 sempre que se desliga o *enable* da saída selecionada. Resultado: `Simulacao OK: descodificador2x4 sem erros`; *bitstream* sem erros (2 LUT).

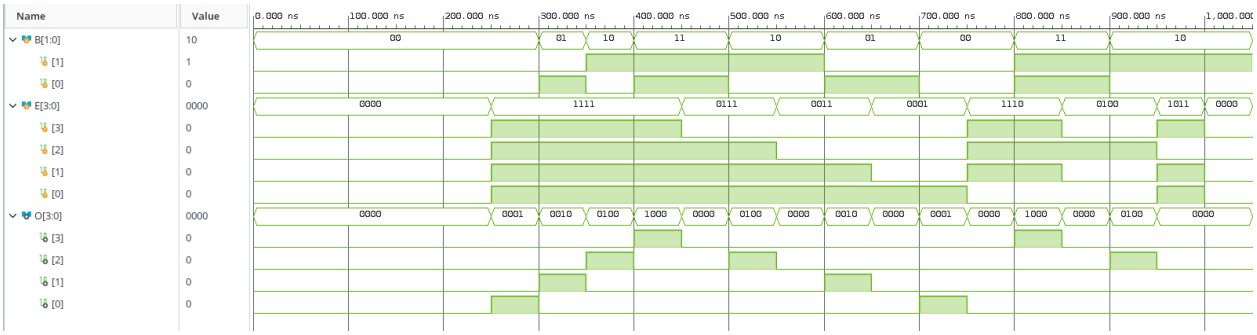


Figura 7: Simulação comportamental do descodificador 2x4 no Vivado.

Na placa (Figura 8): em (a), SW15 SW14 = 01 com SW1 ligado acende só LD1; em (b), 10 com SW2 ligado acende só LD2. Os *enables* das saídas não selecionadas não acendem nada.

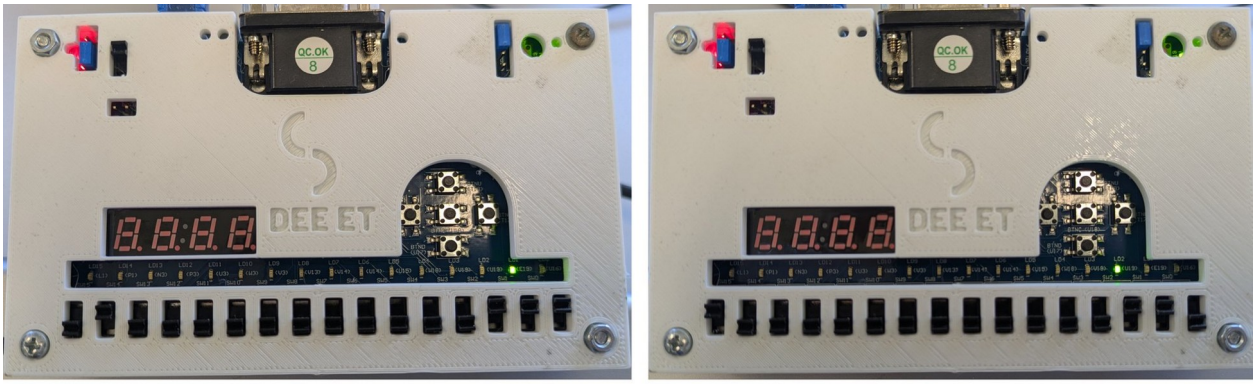


Figura 8: Ensaio do descodificador 2x4 na placa Basys3: (a) saída 1 selecionada, LD1 aceso; (b) saída 2 selecionada, LD2 aceso.

4 Multiplexer 16x4

O *multiplexer* 16x4 (Fig. 27 do enunciado) foi criado no Vivado por **hierarquia de módulos**, ou seja, com uma descrição **estrutural**: quatro *multiplexers* 4x1 com a seleção partilhada, em que `mux`*i* trata o *bit i* das quatro entradas (Listagem 6). A hierarquia (Figura 9) é igual à da Fig. 29 do enunciado.

Listagem 6: Multiplexer 16x4 construído com quatro multiplexers 4x1 (`multiplexer16x4/multiplexer16x4.v`)

```
1  `timescale 1ns / 1ps
2
3  // Multiplexer 16x4: 4 entradas de 4 bits, selecionadas por Sel[1:0].
4  // Construído estruturalmente com 4 multiplexers 4x1, um por cada bit.
5  module multiplexer16x4(
6      input [3:0] DataIn0,
7      input [3:0] DataIn1,
8      input [3:0] DataIn2,
9      input [3:0] DataIn3,
10     output [3:0] DataOut,
11     input [1:0] Sel
12     );
13
14 //   Instanciação do mux4x1 do bit 0 e ligação das entradas e saídas
15     multiplexer4x1 mux0 (
16         .Sel1(Sel[1]),
17         .Sel0(Sel[0]),
18         .DataIn0(DataIn0[0]),
19         .DataIn1(DataIn1[0]),
20         .DataIn2(DataIn2[0]),
21         .DataIn3(DataIn3[0]),
22         .DataOut(DataOut[0])
23     );
24
25 //   Instanciação do mux4x1 do bit 1 e ligação das entradas e saídas
26     multiplexer4x1 mux1 (
27         .Sel1(Sel[1]),
28         .Sel0(Sel[0]),
29         .DataIn0(DataIn0[1]),
30         .DataIn1(DataIn1[1]),
31         .DataIn2(DataIn2[1]),
32         .DataIn3(DataIn3[1]),
33         .DataOut(DataOut[1])
34     );
35
36 //   Instanciação do mux4x1 do bit 2 e ligação das entradas e saídas
37     multiplexer4x1 mux2 (
38         .Sel1(Sel[1]),
39         .Sel0(Sel[0]),
40         .DataIn0(DataIn0[2]),
41         .DataIn1(DataIn1[2]),
42         .DataIn2(DataIn2[2]),
43         .DataIn3(DataIn3[2]),
44         .DataOut(DataOut[2])
45     );
46
47 //   Instanciação do mux4x1 do bit 3 e ligação das entradas e saídas
48     multiplexer4x1 mux3 (
49         .Sel1(Sel[1]),
50         .Sel0(Sel[0]),
51         .DataIn0(DataIn0[3]),
52         .DataIn1(DataIn1[3]),
53         .DataIn2(DataIn2[3]),
54         .DataIn3(DataIn3[3]),
```

```
55          .DataOut(DataOut[3])
56      );
57
58  endmodule
```

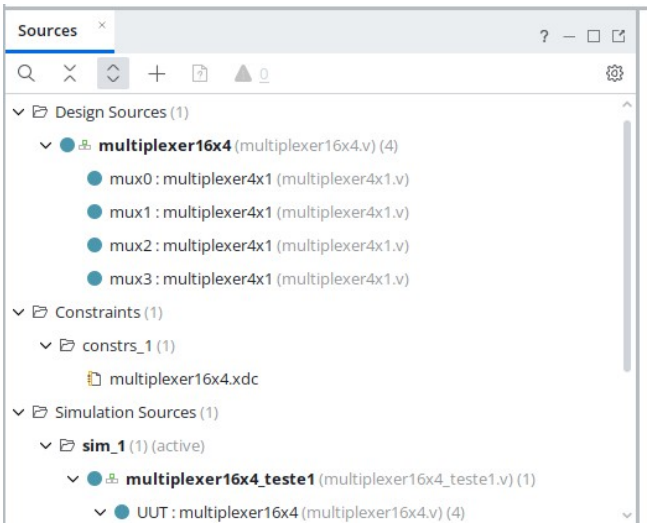


Figura 9: Hierarquia de módulos do multiplexer 16x4 na janela Sources do Vivado.

As entradas usam os 16 interruptores, a seleção duas teclas e a saída os LEDs LD3 a LD0 (Tabela 3 e Listagem 7).

Tabela 3: Ligação dos sinais do multiplexer 16x4 à Basys3.

Sinal	Elemento da Basys3
DataIn0[3:0]	Interruptores SW3 a SW0
DataIn1[3:0]	Interruptores SW7 a SW4
DataIn2[3:0]	Interruptores SW11 a SW8
DataIn3[3:0]	Interruptores SW15 a SW12
Sel[1] / Sel[0]	Teclas BTNL / BTNR (premida = 1)
DataOut[3:0]	LEDs LD3 a LD0

Listagem 7: Ficheiro de restrições do multiplexer 16x4 (`multiplexer16x4/multiplexer16x4.xdc`)

```
1  # Variáveis de seleção do multiplexer 16x4 (teclas BTNL e BTNR)
2  set_property PACKAGE_PIN W19 [get_ports {Sel[1]}]
3      set_property IOSTANDARD LVCMOS33 [get_ports {Sel[1]}]
4  set_property PACKAGE_PIN T17 [get_ports {Sel[0]}]
5      set_property IOSTANDARD LVCMOS33 [get_ports {Sel[0]}]
6
7  # Entrada DataIn0 do multiplexer 16x4 (interruptores SW3 a SW0)
8  set_property PACKAGE_PIN W17 [get_ports {DataIn0[3]}]
9  set_property PACKAGE_PIN W16 [get_ports {DataIn0[2]}]
10 set_property PACKAGE_PIN V16 [get_ports {DataIn0[1]}]
11 set_property PACKAGE_PIN V17 [get_ports {DataIn0[0]}]
12     set_property IOSTANDARD LVCMOS33 [get_ports {DataIn0[*]}]
13
14 # Entrada DataIn1 do multiplexer 16x4 (interruptores SW7 a SW4)
15 set_property PACKAGE_PIN W13 [get_ports {DataIn1[3]}]
16 set_property PACKAGE_PIN W14 [get_ports {DataIn1[2]}]
17 set_property PACKAGE_PIN V15 [get_ports {DataIn1[1]}]
18 set_property PACKAGE_PIN W15 [get_ports {DataIn1[0]}]
19     set_property IOSTANDARD LVCMOS33 [get_ports {DataIn1[*]}]
20
```

```
# Entrada DataIn2 do multiplexer 16x4 (interruptores SW11 a SW8)
set_property PACKAGE_PIN R3 [get_ports {DataIn2[3]}]
set_property PACKAGE_PIN T2 [get_ports {DataIn2[2]}]
set_property PACKAGE_PIN T3 [get_ports {DataIn2[1]}]
set_property PACKAGE_PIN V2 [get_ports {DataIn2[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {DataIn2[*]}]

# Entrada DataIn3 do multiplexer 16x4 (interruptores SW15 a SW12)
set_property PACKAGE_PIN R2 [get_ports {DataIn3[3]}]
set_property PACKAGE_PIN T1 [get_ports {DataIn3[2]}]
set_property PACKAGE_PIN U1 [get_ports {DataIn3[1]}]
set_property PACKAGE_PIN W2 [get_ports {DataIn3[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {DataIn3[*]}]

# Saída do multiplexer 16x4 (LEDs LD3 a LD0)
set_property PACKAGE_PIN V19 [get_ports {DataOut[3]}]
set_property PACKAGE_PIN U19 [get_ports {DataOut[2]}]
set_property PACKAGE_PIN E19 [get_ports {DataOut[1]}]
set_property PACKAGE_PIN U16 [get_ports {DataOut[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {DataOut[*]}]

# Definir a tensão correta da placa Basys3
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

O *testbench* (Listagem 18, no Anexo A) coloca A, 5, C e 3 nas entradas e percorre as seleções; depois muda uma entrada não selecionada (a saída não muda) e a selecionada (a saída acompanha-a) e aplica 1, 2, 4 e 8 para confirmar a ligação de cada *bit*. Na Figura 10 a saída é sempre igual à entrada selecionada. Resultado: `Simulacao OK: multiplexer16x4 sem erros`; *bitstream* sem erros (4 LUT, uma por *bit*).

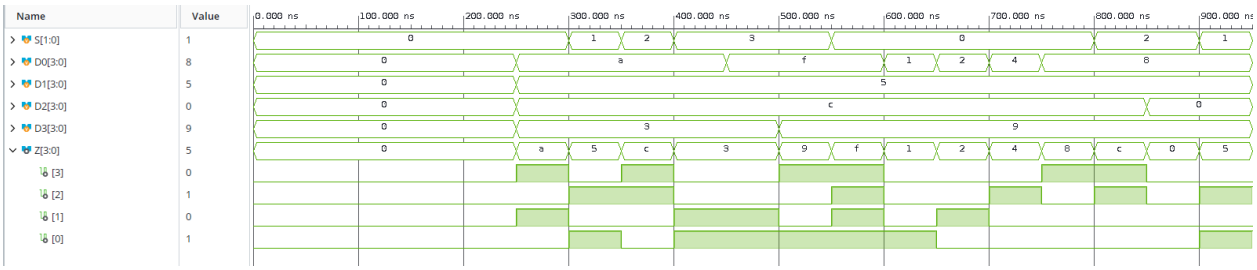


Figura 10: Simulação comportamental do multiplexer 16x4 no Vivado.

Na placa (Figura 11) cada grupo de interruptores tem um valor e as teclas escolhem o grupo mostrado nos LEDs: (a) com BTNR premida, 1010 (DataIn1); (b) com as duas teclas, 1001 (DataIn3).

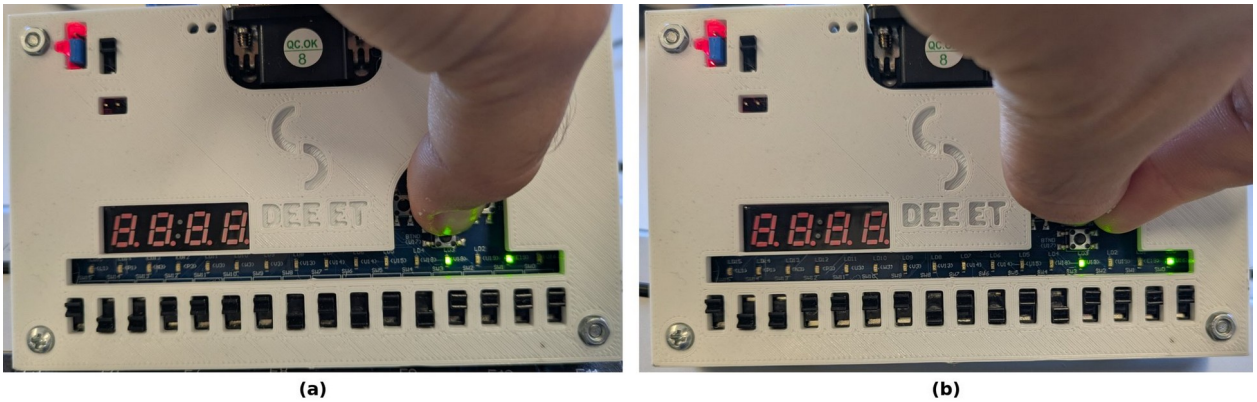


Figura 11: Ensaio do multiplexer 16x4 na placa Basys3: (a) BTNR premida, entrada DataIn1; (b) BTNL e BTNR premidas, entrada DataIn3.

5 Transcodificador BCD para display de 7 segmentos

O transcodificador `Transcod7Seg` converte `Numero[3:0]` em `Segmentos[6:0]` (índice 6 = segmento *a*, índice 0 = segmento *g*), com saídas **ativas a Low**. Para 0 a 9 usaram-se os algarismos da Fig. 30 do enunciado; para 10 a 15 criaram-se símbolos originais (Tabela 4 e Figura 12): as iniciais do grupo, **r** e **d**, as letras **L**, **E** e **C** (de LEEC) e um traço, escolhidos para não se confundirem com nenhum algarismo (por exemplo, o "d" é minúsculo porque o "D" seria igual ao 0).

Tabela 4: Configuração dos segmentos para cada valor de Numero (0 = segmento aceso).

Numero	Segmentos[6:0] (a b c d e f g)	Segmentos acesos	Símbolo
0	`0000001`	a b c d e f	0
1	`1001111`	b c	1
2	`0010010`	a b d e g	2
3	`0000110`	a b c d g	3
4	`1001100`	b c f g	4
5	`0100100`	a c d f g	5
6	`0100000`	a c d e f g	6
7	`0001111`	a b c	7
8	`0000000`	a b c d e f g	8
9	`0000100`	a b c d f g	9
10	`1111010`	e g	r
11	`1000010`	b c d e g	d
12	`1110001`	d e f	L
13	`0110000`	a d e f g	E
14	`0110001`	a d e f	C
15	`1111110`	g	-

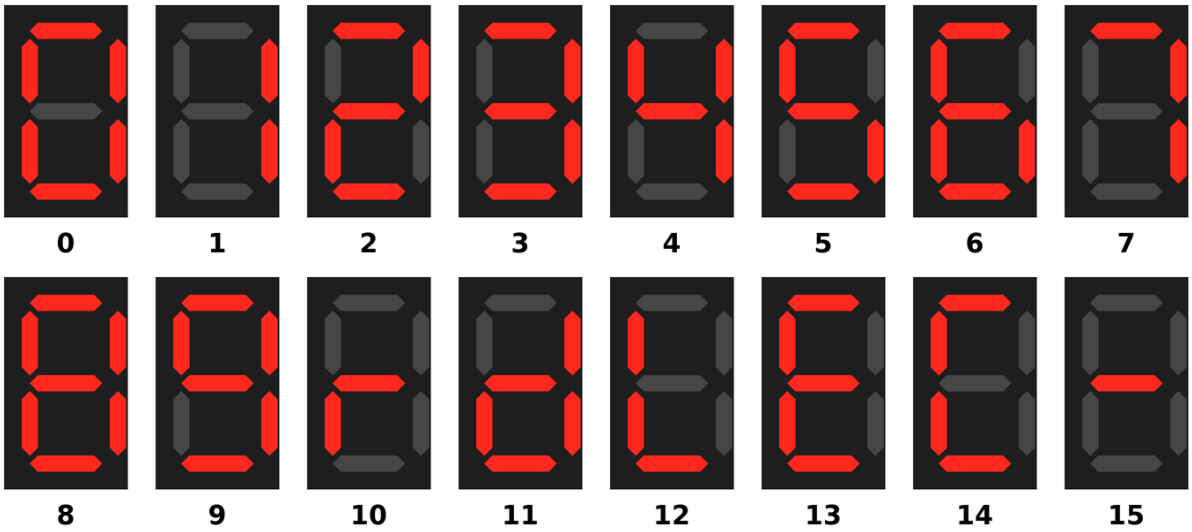


Figura 12: Os 16 símbolos do transcodificador, desenhados a partir dos valores de `transcod7seg.v` (a vermelho os segmentos acesos).

O módulo (Listagem 8) foi adaptado do exemplo do Moodle, cujos erros são analisados na secção 9.2. Sendo combinatório, a saída é `output reg` atribuída num `always @(*)` com atribuições bloqueantes.

Listagem 8: Transcodificador para display de 7 segmentos (adaptado do exemplo do Moodle) (`transcod7seg/transcod7seg.v`)

```
1  `timescale 1ns / 1ps
```

```
2
3  // Transcodificador de BCD (e símbolos 10 a 15) para display de 7 segmentos.
4  // Segmentos[6] = a ... Segmentos[0] = g, ativos a Low (0 = segmento aceso).
5  // O ponto decimal não é tratado neste módulo.
6  // Adaptado do módulo Transcod7Seg dos exemplos Verilog do Moodle,
7  // com os erros corrigidos e símbolos 10 a 15 originais (ver relatório, secção 9.2).
8  module Transcod7Seg(
9      input [3:0] Numero,
10     output reg [6:0] Segmentos
11     );
12
13      always @(*)
14          case (Numero)               //  abcdefg
15              4'd0:  Segmentos = 7'b0000001;   // 0
16              4'd1:  Segmentos = 7'b1001111;   // 1
17              4'd2:  Segmentos = 7'b0010010;   // 2
18              4'd3:  Segmentos = 7'b0000110;   // 3
19              4'd4:  Segmentos = 7'b1001100;   // 4
20              4'd5:  Segmentos = 7'b0100100;   // 5
21              4'd6:  Segmentos = 7'b0100000;   // 6
22              4'd7:  Segmentos = 7'b0001111;   // 7
23              4'd8:  Segmentos = 7'b0000000;   // 8
24              4'd9:  Segmentos = 7'b0000100;   // 9
25              // Símbolos originais nas combinações não BCD: iniciais do grupo (r, d),
26              // LEC (de LEEC) e um traço: "rdLEC-"
27              4'd10: Segmentos = 7'b1111010;   // r (e g)
28              4'd11: Segmentos = 7'b1000010;   // d (b c d e g)
29              4'd12: Segmentos = 7'b1110001;   // L (d e f)
30              4'd13: Segmentos = 7'b0110000;   // E (a d e f g)
31              4'd14: Segmentos = 7'b0110001;   // C (a d e f)
32              4'd15: Segmentos = 7'b1111110;   // - (g)
33              default: Segmentos = 7'b1111111; // tudo apagado (nunca usado: os 16 casos estão cobertos)
34         endcase
35
36 endmodule
```

O *testbench* (Listagem 19, no Anexo A) compara a saída com uma tabela de referência dos segmentos acesos, verifica que não há segmentos indefinidos e que os 16 símbolos são diferentes, e repete os valores por ordem aleatória (48 verificações). Na Figura 13 os *bits* de `S` coincidem para cada `N` com a Tabela 4 (por exemplo, todos a 0 para N = 8). Resultado: `Simulacao OK: Transcod7Seg sem erros (48 verificacoes)`.

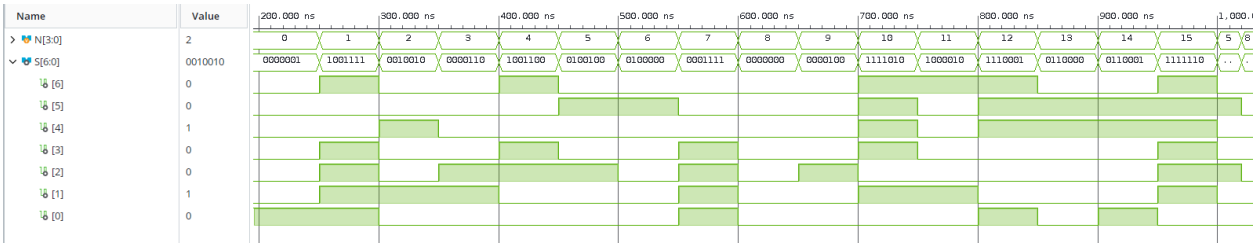


Figura 13: Simulação comportamental do transcodificador no Vivado (200 a 1000 ns).

5.1 Ensaio do transcodificador na placa

Para o ensaio na placa criou-se o módulo de topo `displayBCD` (Listagem 9, Tabela 5 e Listagem 10): o valor vem de SW3 a SW0 e SW15 a SW12 escolhem os *displays* que acendem (negados, porque os ânodos são ativos a *Low*); o ponto decimal fica apagado.

Listagem 9: Circuito de ensaio do transcodificador (`transcod7seg/displayBCD.v`)

```
1  `timescale 1ns / 1ps
```

```
2
3  // Circuito de ensaio do transcodificador na Basys3.
4  // SW3..SW0 escolhem o símbolo e SW15..SW12 escolhem quais displays acendem.
5  // Os ânodos são comuns e ativos a Low, por isso inverte-se o valor dos interruptores.
6  module displayBCD(
7      input [3:0] Numero,
8      input [3:0] Liga,
9      output [6:0] Segmentos,
10     output DotPoint,
11     output [3:0] Anodos
12     );
13
14      Transcod7Seg transcod (
15          .Numero(Numero),
16          .Segmentos(Segmentos)
17      );
18
19      assign Anodos = ~Liga;
20      assign DotPoint = 1'b1;       // ponto decimal apagado
21
22 endmodule
```

Tabela 5: Ligação dos sinais do circuito `displayBCD` à Basys3.

Sinal	Pinos da FPGA	Elemento da Basys3
Numero[3:0]	W17 W16 V16 V17	Interruptores SW3 a SW0
Liga[3:0]	R2 T1 U1 W2	Interruptores SW15 a SW12
Segmentos[6:0]	W7 W6 U8 V8 U5 V5 U7	Segmentos CA a CG (ativos a Low)
DotPoint	V7	Ponto decimal DP (ativo a Low)
Anodos[3:0]	W4 V4 U4 U2	Ânodos AN3 a AN0 (ativos a Low)

Listagem 10: Ficheiro de restrições do circuito `displayBCD` (`transcod7seg/displayBCD.xdc`)

```
1  # Ensaio do transcodificador para 7 segmentos (módulo de topo displayBCD)
2
3  # Número a apresentar (interruptores SW3 a SW0)
4  set_property PACKAGE_PIN W17 [get_ports {Numero[3]}]
5  set_property PACKAGE_PIN W16 [get_ports {Numero[2]}]
6  set_property PACKAGE_PIN V16 [get_ports {Numero[1]}]
7  set_property PACKAGE_PIN V17 [get_ports {Numero[0]}]
8      set_property IOSTANDARD LVCMOS33 [get_ports {Numero[*]}]
9
10 # Displays a ligar (interruptores SW15 a SW12, um por display)
11 set_property PACKAGE_PIN R2 [get_ports {Liga[3]}]
12 set_property PACKAGE_PIN T1 [get_ports {Liga[2]}]
13 set_property PACKAGE_PIN U1 [get_ports {Liga[1]}]
14 set_property PACKAGE_PIN W2 [get_ports {Liga[0]}]
15     set_property IOSTANDARD LVCMOS33 [get_ports {Liga[*]}]
16
17 # Segmentos a..g (Segmentos[6] = a ... Segmentos[0] = g), ativos a Low
18 set_property PACKAGE_PIN W7 [get_ports {Segmentos[6]}]
19 set_property PACKAGE_PIN W6 [get_ports {Segmentos[5]}]
20 set_property PACKAGE_PIN U8 [get_ports {Segmentos[4]}]
21 set_property PACKAGE_PIN V8 [get_ports {Segmentos[3]}]
22 set_property PACKAGE_PIN U5 [get_ports {Segmentos[2]}]
23 set_property PACKAGE_PIN V5 [get_ports {Segmentos[1]}]
24 set_property PACKAGE_PIN U7 [get_ports {Segmentos[0]}]
25     set_property IOSTANDARD LVCMOS33 [get_ports {Segmentos[*]}]
26
```

```
# Ponto decimal
set_property PACKAGE_PIN V7 [get_ports DotPoint]
    set_property IOSTANDARD LVCMOS33 [get_ports DotPoint]

# Ânodos comuns dos 4 displays (ativos a Low); Anodos[0] é o display da direita
set_property PACKAGE_PIN U2 [get_ports {Anodos[0]}]
set_property PACKAGE_PIN U4 [get_ports {Anodos[1]}]
set_property PACKAGE_PIN V4 [get_ports {Anodos[2]}]
set_property PACKAGE_PIN W4 [get_ports {Anodos[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Anodos[*]}]

# Definir a tensão correta da placa Basys3
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

O *bitstream* foi gerado sem erros (8 LUT). Na Figura 14: em (a), só com SW12 e SW3 a SW0 = 1011, o *display* da direita mostra "d"; em (b), com SW15, SW13 e SW12 e o valor 0101, três *displays* mostram 5 e o desligado fica apagado.

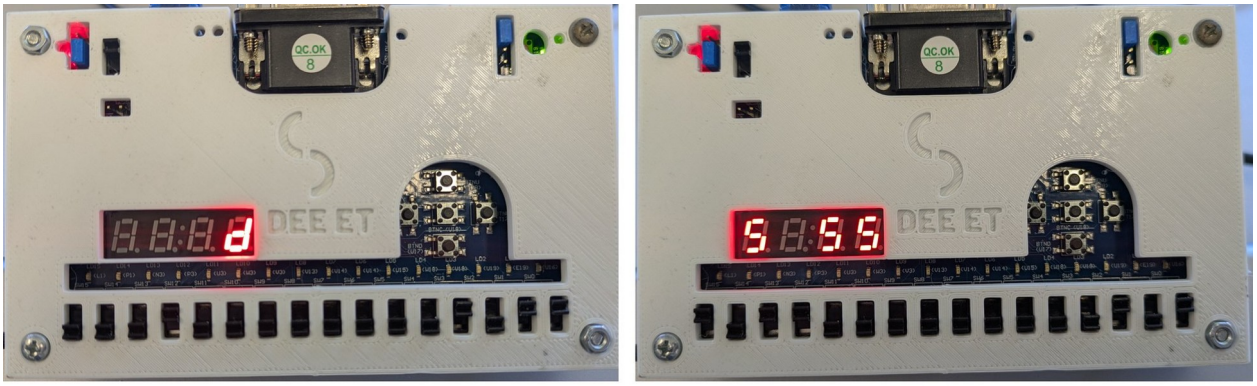


Figura 14: Ensaio do transcodificador na placa Basys3: (a) símbolo "d" no display da direita; (b) algarismo 5 em três displays.

6 Registo de 4 bits com clear e enable

O `Registo4` (Listagem 11) usa quatro primitivas **FDCE** da Xilinx, uma por *bit*: *flip-flop* D sensível ao flanco ascendente, com *clock enable* (`CE`) e *clear* assíncrono (`CLR`, ativo a *High*, com prioridade sobre a carga). Com `CLR = 0` a saída copia `D` no flanco ascendente se `CE = 1` e mantém o valor se `CE = 0`; `INIT = 0` define o valor inicial. A instanciação segue o modelo do Vivado (Tools > Language Templates).

Listagem 11: Registo de 4 bits com quatro primitivas FDCE (`registo4/registo4.v`)

```
1  `timescale 1ns / 1ps
2
3  // Registo de 4 bits com clear assíncrono e clock enable.
4  // Feito com 4 primitivas FDCE (flip-flop D com CLR assíncrono e CE),
5  // sensíveis ao flanco ascendente do relógio.
6  module Registo4(
7      input [3:0] Data,
8      input CE,
9      input Clk,
10     input CLR,
11     output [3:0] DataOut
12     );
13
14 //   Instanciação dos 4 flip-flops FDCE, um por cada bit
15      FDCE #(.INIT(1'b0)) ff0 (.Q(DataOut[0]), .C(Clk), .CE(CE), .CLR(CLR), .D(Data[0]));
16      FDCE #(.INIT(1'b0)) ff1 (.Q(DataOut[1]), .C(Clk), .CE(CE), .CLR(CLR), .D(Data[1]));
17      FDCE #(.INIT(1'b0)) ff2 (.Q(DataOut[2]), .C(Clk), .CE(CE), .CLR(CLR), .D(Data[2]));
18      FDCE #(.INIT(1'b0)) ff3 (.Q(DataOut[3]), .C(Clk), .CE(CE), .CLR(CLR), .D(Data[3]));
19
20 endmodule
```

O *testbench* (Listagem 20, no Anexo A), com relógio de 100 MHz, tem casos dirigidos (valor inicial, carga só com `CE = 1`, *clear* assíncrono a meio de um ciclo e com prioridade sobre `CE`, todos os valores) e 2000 ciclos com entradas aleatórias, comparados com um modelo de referência.

Na Figura 15, com `CE = 0` a saída `Q` não muda; com `CE = 1` carrega 5 e A nos flancos de 115 e 125 ns. Aos 183 ns, entre dois flancos, o `CLR` põe `Q` imediatamente a 0, mesmo com `CE = 1`, e só no flanco seguinte (205 ns) o registo carrega o F. O modelo de referência coincide sempre com `Q`. Resultado: `Simulacao OK: Registo4 sem erros (3056 verificacoes)`.

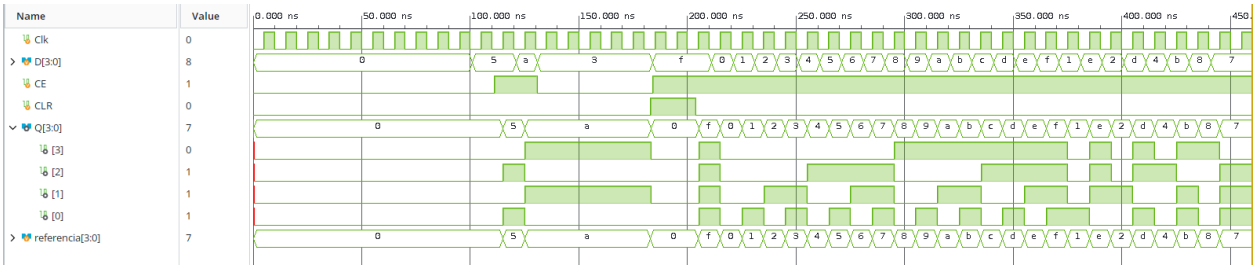


Figura 15: Simulação do registo de 4 bits no Vivado: casos dirigidos (0 a 460 ns).

7 Divisor de clock e contador de módulo 4

7.1 Divisor de clock

O `DivisorClock` (Listagem 12) reduz os 100 MHz da placa para o varrimento: conta ciclos de `ClockIn` e inverte `ClockOut` ao fim de `METADE` ciclos (*duty cycle* de 50 %):

$$f_out = f_in / (2 \times METADE) = 100\ MHz / (2 \times 25\,000) = 2\ kHz$$

`METADE = 25000` dá 2 kHz, dentro do intervalo pedido (1 a 10 kHz). Por ser um parâmetro, pode ser reduzido na simulação ou aumentado na placa para *debug* visual (25 000 000 dá 2 Hz). O módulo simplifica os divisores do Moodle (secção 9.3).

Listagem 12: Divisor de clock (baseado nos exemplos do Moodle) (`divisorclock/divisorclock.v`)

```
1  `timescale 1ns / 1ps
2
3  // Divisor de clock: gera ClockOut a partir do relógio de 100 MHz da Basys3.
4  // ClockOut inverte a cada METADE ciclos de ClockIn, logo
5  // f_out = f_in / (2 * METADE) = 100 MHz / 50000 = 2 kHz (entre 1 e 10 kHz).
6  // Para debug visual (displays a piscar) usar, por exemplo, METADE = 25_000_000 (2 Hz).
7  // Baseado nos divisores clk_div e Clock_Divider dos exemplos Verilog do Moodle,
8  // sem enable e reset e com a contagem corrigida (relatório, secção 9.3).
9  module DivisorClock #(
10     parameter METADE = 25000
11     )(
12     input ClockIn,
13     output reg ClockOut = 1'b0
14     );
15
16      reg [31:0] contagem = 0;
17
18      always @(posedge ClockIn)
19          if (contagem == METADE - 1) begin
20              contagem <= 0;
21              ClockOut <= ~ClockOut;
22          end
23          else
24              contagem <= contagem + 1;
25
26 endmodule
```

O *testbench* (Listagem 21, no Anexo A) usa `METADE` = 1, 2, 5 e 7 e confirma que a saída fica exatamente `METADE` ciclos a 1 e a 0, mudando só nos flancos ascendentes; uma instância com 25 000 confirma os 2000 Hz. Na Figura 16 `Out1` muda a cada ciclo e `Out7` a cada 7 ciclos, com o contador interno a contar de 0 a 6. Resultado: `Simulacao OK: DivisorClock sem erros.`

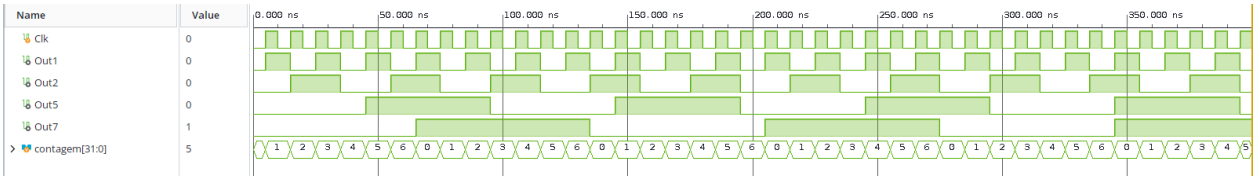


Figura 16: Simulação do divisor de clock no Vivado para METADE = 1, 2, 5 e 7 (0 a 400 ns).

7.2 Contador de módulo 4

O `Contador4` (Listagem 13) é um registo de 2 *bits* que soma 1 em cada flanco ascendente e volta naturalmente a 0 depois de 3; foi obtido do "contador simples" do Moodle (secção 9.4). As saídas `Q1` e `Q0` são as variáveis de varrimento `Control1` e `Control0` da Fig. 31: a 2 kHz, cada *display* fica ativo 0,5 ms e, por persistência da imagem na retina, os quatro parecem sempre acesos.

Listagem 13: Contador de módulo 4 (baseado no exemplo do Moodle) (`contador4/contador4.v`)

```
1  `timescale 1ns / 1ps
2
3  // Contador de módulo 4 (0, 1, 2, 3, 0, ...) que avança no flanco ascendente de Clock.
4  // Q1 Q0 vão atuar nas seleções dos multiplexers e nas entradas do descodificador.
5  // Baseado no "Contador simples" dos exemplos Verilog do Moodle,
6  // reduzido a 2 bits com atribuição não bloqueante (relatório, secção 9.4).
7  module Contador4(
8      input Clock,
9      output Q1,
10     output Q0
11     );
12
13      reg [1:0] estado = 2'b00;
14
15      always @(posedge Clock)
16          estado <= estado + 1;     // 3 + 1 = 0 em 2 bits: módulo 4
17
18      assign Q1 = estado[1];
19      assign Q0 = estado[0];
20
21 endmodule
```

O *testbench* (Listagem 22, no Anexo A) verifica o estado inicial e a sequência durante 100 flancos. Na Figura 17 vê-se 0, 1, 2, 3, 0, …, com `Q0` a mudar em todos os flancos ascendentes e `Q1` a cada dois. Resultado: `Simulacao OK: Contador4 sem erros (100 ciclos).`

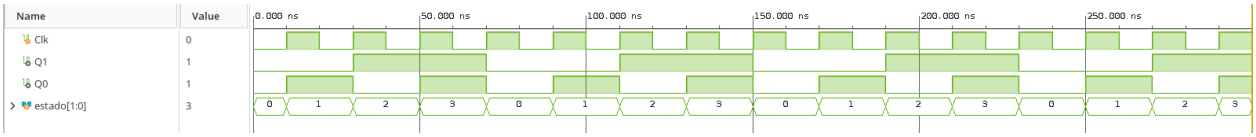


Figura 17: Simulação do contador de módulo 4 no Vivado (0 a 300 ns).

8 Controlador dos displays da placa Basys3

O `ControloDisplays` (Listagem 14) integra, sem alterações, os módulos anteriores, conforme a Fig. 31 do enunciado (Tabela 6).

Tabela 6: Módulos que constituem o controlador dos displays.

Instância	Módulo	Função
reg0 a reg3	Registo4	Guardam o código do símbolo de cada *display* (carga com Load*i*, *clear* com Clr*i*)
regDP	Registo4	Guarda o estado dos quatro pontos decimais (1 = ponto aceso)
divisor	DivisorClock	Reduz a frequência de 100 MHz para 2 kHz (relógio do varrimento)
contador	Contador4	Gera as variáveis de varrimento Control1 e Control0 (0, 1, 2, 3, 0, …)
mux16x4	multiplexer16x4	Liga ao transcodificador o registo do *display* ativo
transcod	Transcod7Seg	Converte o símbolo nos segmentos a acender (partilhados por todos os *displays*)
mux4x1	multiplexer4x1	Escolhe o ponto decimal do *display* ativo
descod	descodificador2x4	Ativa o ânodo do *display* selecionado, se o respetivo Enable estiver a 1

Na integração houve três cuidados:

1. **Ânodos ativos a Low.** Na Basys3 cada ânodo é alimentado por um transístor PNP; ligar o descodificador diretamente, como na Fig. 31, acenderia os três *displays* não selecionados. Por isso as suas saídas são negadas (`assign Anodos = ~DisplayAtivo`).
2. **Ponto decimal.** O registo dos pontos guarda 1 para "aceso" e a saída do *multiplexer* 4x1 é negada, porque o pino DP também é ativo a *Low*.
3. **Nomes das portas.** As ligações usam os nomes reais dos módulos reutilizados (`DataIn0`…`DataIn3`, `Sel`, `En0`…`En3`) em vez dos da Fig. 31; o registo *i* liga à entrada *i* do *multiplexer* 16x4, pelo que `Anodos[i]` mostra sempre o registo *i*.

Os registos usam o relógio de 100 MHz e o contador o relógio dividido; `METADE` é passado ao divisor para poder ser reduzido nos *testbenches*.

Listagem 14: Controlador dos quatro displays (Fig. 31) (`controlodisplays/controlodisplays.v`)

```
1  `timescale 1ns / 1ps
2
3  // Controlador dos 4 displays de 7 segmentos da Basys3 (Fig. 31 do enunciado).
4  // Reutiliza os módulos já desenvolvidos:
5  //   - 5 x Registo4: 4 registos com o símbolo de cada display e 1 com os pontos decimais;
6  //   - multiplexer16x4: liga ao transcodificador o registo do display ativo;
7  //   - multiplexer4x1: escolhe o ponto decimal do display ativo;
8  //   - Transcod7Seg: converte o símbolo nos segmentos a acender;
9  //   - descodificador2x4: escolhe o display ativo (com um enable por display);
10 //   - DivisorClock + Contador4: geram as variáveis de varrimento Control1 e Control0.
11 // O display i (Anodos[i]) mostra o registo i; Anodos[0] é o display da direita.
12 // Segmentos, ponto decimal e ânodos são ativos a Low na Basys3.
13 module ControloDisplays #(
14     parameter METADE = 25000        // varrimento a 100 MHz / (2 x 25000) = 2 kHz
15     )(
16     input [3:0] Data3,
17     input Load3,
18     input Clr3,
19     input Enable3,
20     input [3:0] Data2,
21     input Load2,
22     input Clr2,
23     input Enable2,
24     input [3:0] Data1,
```

```
    input Load1,
    input Clr1,
    input Enable1,
    input [3:0] Data0,
    input Load0,
    input Clr0,
    input Enable0,
    input [3:0] DotPoints,
    input LoadDP,
    input ClrDP,
    input Clock,
    output [6:0] Segmentos,
    output DotPoint,
    output [3:0] Anodos
    );

//   Sinais internos
     wire [3:0] Valor3, Valor2, Valor1, Valor0;   // conteúdo dos registos dos símbolos
     wire [3:0] Pontos;                           // conteúdo do registo dos pontos decimais
     wire [3:0] ValorAtivo;                       // símbolo do display ativo
     wire PontoAtivo;                             // ponto decimal do display ativo (1 = aceso)
     wire [3:0] DisplayAtivo;                     // saídas do descodificador (ativas a High)
     wire ClockVarrimento;                        // relógio dividido
     wire Control1, Control0;                     // variáveis de varrimento

//   Registos dos símbolos e dos pontos decimais (relógio de 100 MHz, carga com Load)
     Registo4 reg3  (.Data(Data3),     .CE(Load3),  .Clk(Clock), .CLR(Clr3),  .DataOut(Valor3));
     Registo4 reg2  (.Data(Data2),     .CE(Load2),  .Clk(Clock), .CLR(Clr2),  .DataOut(Valor2));
     Registo4 reg1  (.Data(Data1),     .CE(Load1),  .Clk(Clock), .CLR(Clr1),  .DataOut(Valor1));
     Registo4 reg0  (.Data(Data0),     .CE(Load0),  .Clk(Clock), .CLR(Clr0),  .DataOut(Valor0));
     Registo4 regDP (.Data(DotPoints), .CE(LoadDP), .Clk(Clock), .CLR(ClrDP), .DataOut(Pontos));

//   Varrimento: divisor de clock e contador de módulo 4
     DivisorClock #(.METADE(METADE)) divisor (
         .ClockIn(Clock),
         .ClockOut(ClockVarrimento)
     );

     Contador4 contador (
         .Clock(ClockVarrimento),
         .Q1(Control1),
         .Q0(Control0)
     );

//   Multiplexer 16x4: registo do display ativo para o transcodificador
     multiplexer16x4 mux16x4 (
         .DataIn0(Valor0),
         .DataIn1(Valor1),
         .DataIn2(Valor2),
         .DataIn3(Valor3),
         .DataOut(ValorAtivo),
         .Sel({Control1, Control0})
     );

//   Transcodificador: segmentos partilhados por todos os displays (ativos a Low)
     Transcod7Seg transcod (
         .Numero(ValorAtivo),
         .Segmentos(Segmentos)
     );

//   Multiplexer 4x1: ponto decimal do display ativo
     multiplexer4x1 mux4x1 (
```

```
87          .Sel1(Control1),
88          .Sel0(Control0),
89          .DataIn0(Pontos[0]),
90          .DataIn1(Pontos[1]),
91          .DataIn2(Pontos[2]),
92          .DataIn3(Pontos[3]),
93          .DataOut(PontoAtivo)
94      );
95
96      assign DotPoint = ~PontoAtivo;     // o segmento DP acende com nível Low
97
98  //  Descodificador 2x4: ativa um display de cada vez, se o respetivo enable estiver a 1
99      descodificador2x4 descod (
100         .Bit1(Control1),
101         .Bit0(Control0),
102         .En0(Enable0),
103         .En1(Enable1),
104         .En2(Enable2),
105         .En3(Enable3),
106         .Out0(DisplayAtivo[0]),
107         .Out1(DisplayAtivo[1]),
108         .Out2(DisplayAtivo[2]),
109         .Out3(DisplayAtivo[3])
110     );
111
112     assign Anodos = ~DisplayAtivo;     // ânodos comuns ligados por transistores PNP: ativos a Low
113
114 endmodule
```

O *testbench* (Listagem 23, no Anexo A) é curto e serve para ver o funcionamento nas formas de onda: com `METADE = 1` cada *display* fica aceso 20 ns e o varrimento demora 80 ns. Cada passo dura 100 ns e está comentado com o que deve ver-se nos *displays*:

- "0000" depois de ligar os *enables*; carga dos quatro registos ("2026") e dos pontos com 0100 ("20.26");
- `Load0` mantido a 1 enquanto `D0` passa por 5, 6 e 7: o registo acompanha `D0` e fica com o 7 quando `Load0` volta a 0 ("20.27");
- carga só dos registos 3 e 1 (`Load` = 1010): "r0.L7";
- *display* 2 desligado ("r L7"), *clear* assíncrono do registo 3 ("0 L7"), pontos apagados ("00L7"), registos a 0 ("0000") e *displays* desligados.

A Figura 18 mostra também o conteúdo dos registos (sinais internos `Valor3` a `Valor0` e `Pontos`), que só muda num flanco ascendente de `Clk` com o respetivo `Load` a 1 ou com o *clear*. Os registos guardam códigos: a, b, c, d, e e f são r, d, L, E, C e - (Tabela 4).

`DotPoints[3:0]` indica, com um *bit* por *display*, os pontos a acender (1 = aceso) e fica guardado em `Pontos` com `LoadDP`. A saída `DotPoint` tem um só *bit*, porque a Basys3 tem um único pino DP, comum aos quatro *displays*: o *multiplexer* 4x1 escolhe o *bit* do *display* ativo, negado porque o pino é ativo a *Low*.

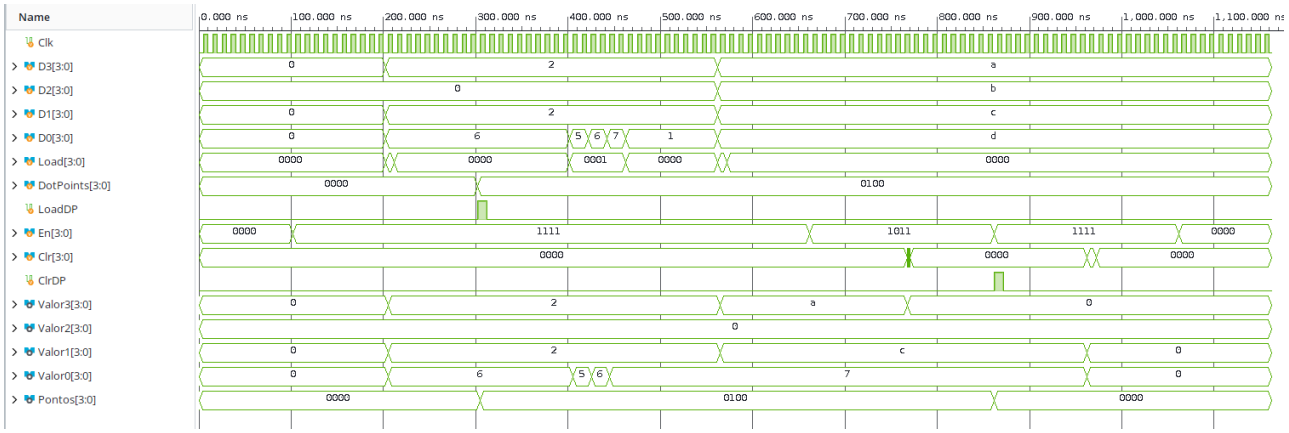


Figura 18: Simulação do controlador dos displays no Vivado (METADE = 1).

Na Figura 19 (um varrimento com "20.26") `Sel` avança a cada 2 ciclos de `Clk` e `Anodos` tem um único *bit* a 0 (1110 para o *display* 0, …, 0111 para o 3). `ValorAtivo` é o código do *display* ativo e `Segmentos` o respetivo símbolo (0100000 = 6, 0010010 = 2, 0000001 = 0); `DotPoint` só vai a 0 no *display* 2. As formas de onda correspondem aos comentários do *testbench*.

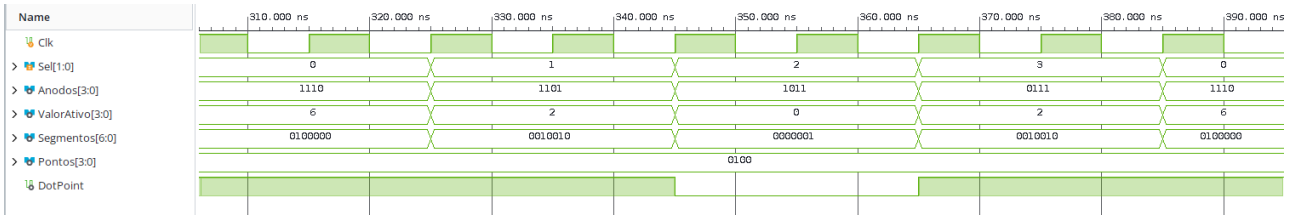


Figura 19: Pormenor da simulação do controlador: um varrimento completo com "20.26".

8.1 Circuito de teste na placa

O `TesteDisplays` (Fig. 32 do enunciado e Listagem 15) liga o mesmo barramento `Data` aos cinco registos: `LD3` a `LD0` e `LDDP` escolhem o registo a carregar, `Clear` limpa todos e `EN` liga ou desliga os quatro *displays*.

Listagem 15: Circuito de teste do controlador na Basys3 (Fig. 32) (`controlodisplays/testedisplays.v`)

```
1  `timescale 1ns / 1ps
2
3  // Circuito de teste do controlador dos displays na Basys3 (Fig. 32 do enunciado).
4  // O mesmo barramento Data alimenta os 4 registos dos símbolos e o registo dos pontos
5  // decimais; cada registo só carrega quando o respetivo sinal LDx está a 1.
6  // Clear limpa todos os registos e EN liga ou desliga os 4 displays.
7  module TesteDisplays #(
8      parameter METADE = 25000        // passado ao divisor de clock do controlador
9      )(
10     input [3:0] Data,
11     input LD3,
12     input LD2,
13     input LD1,
14     input LD0,
15     input LDDP,
16     input EN,
17     input Clear,
18     input CLKBasys,
19     output [6:0] Seg,
20     output DP,
21     output [3:0] An
22     );
23
24      ControloDisplays #(.METADE(METADE)) controlo (
25          .Data3(Data),  .Load3(LD3),  .Clr3(Clear),  .Enable3(EN),
```

```
        .Data2(Data),  .Load2(LD2),  .Clr2(Clear),  .Enable2(EN),
        .Data1(Data),  .Load1(LD1),  .Clr1(Clear),  .Enable1(EN),
        .Data0(Data),  .Load0(LD0),  .Clr0(Clear),  .Enable0(EN),
        .DotPoints(Data),
        .LoadDP(LDDP),
        .ClrDP(Clear),
        .Clock(CLKBasys),
        .Segmentos(Seg),
        .DotPoint(DP),
        .Anodos(An)
    );

endmodule
```

Nos pinos (Tabela 7 e Listagem 16) o valor fica em SW3 a SW0 e os interruptores de carga SW15 a SW12 seguem a ordem dos *displays*; com o interruptor ligado, o *display* acompanha `Data` e, ao desligá-lo, fica com o último valor. O *clear* ficou na tecla BTNC.

Tabela 7: Ligação dos sinais do circuito de teste à Basys3.

Sinal	Pino(s)	Elemento da Basys3
CLKBasys	W5	Oscilador de 100 MHz
Data[3:0]	W17 W16 V16 V17	Interruptores SW3 a SW0
LD3 / LD2 / LD1 / LD0	R2 / T1 / U1 / W2	Interruptores SW15 / SW14 / SW13 / SW12
LDDP	R3	Interruptor SW11
EN	T2	Interruptor SW10
Clear	U18	Tecla BTNC
Seg[6:0]	W7 W6 U8 V8 U5 V5 U7	Segmentos CA a CG (ativos a Low)
DP	V7	Ponto decimal (ativo a Low)
An[3:0]	W4 V4 U4 U2	Ânodos AN3 a AN0 (ativos a Low)

As restrições definem também o relógio de 100 MHz e o relógio gerado pelo divisor (divisão por 50 000). O aviso crítico TIMING-8 inicial, causado pela realimentação de `ClockOut` entre os dois relógios, foi resolvido declarando esse caminho como falso (`set_false_path`), tal como as entradas e saídas, que não têm requisitos temporais. A implementação cumpre todas as restrições (WNS de 5,020 ns).

Listagem 16: Ficheiro de restrições do circuito de teste (`controlodisplays/testedisplays.xdc`)

```
# Ensaio do controlador dos displays na Basys3 (módulo de topo TesteDisplays, Fig. 32)

# Relógio interno da Basys3: 100 MHz
set_property PACKAGE_PIN W5 [get_ports CLKBasys]
    set_property IOSTANDARD LVCMOS33 [get_ports CLKBasys]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLKBasys]

# Relógio do varrimento, gerado pelo divisor (100 MHz / 50000 = 2 kHz)
create_generated_clock -name clk_varrimento -source [get_ports CLKBasys] -divide_by 50000 [get_pins
controlo/divisor/ClockOut_reg/Q]
# O único caminho entre os dois relógios é a realimentação do próprio divisor
# (ClockOut_reg/Q -> inversor -> ClockOut_reg/D), sem período comum que o Vivado consiga analisar
set_false_path -from [get_clocks clk_varrimento] -to [get_clocks sys_clk_pin]

# Interruptores, teclas e displays são lentos e assíncronos em relação ao relógio:
# não há requisitos de tempo nas entradas e saídas
set_false_path -from [get_ports {Data[*] LD3 LD2 LD1 LD0 LDDP EN Clear}]
set_false_path -to [get_ports {Seg[*] DP An[*]}]

```

```
# Valor a carregar nos registos (interruptores SW3 a SW0)
set_property PACKAGE_PIN W17 [get_ports {Data[3]}]
set_property PACKAGE_PIN W16 [get_ports {Data[2]}]
set_property PACKAGE_PIN V16 [get_ports {Data[1]}]
set_property PACKAGE_PIN V17 [get_ports {Data[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Data[*]}]

# Carga dos registos dos símbolos (interruptores SW15 a SW12, alinhados com os displays:
# SW15 carrega o display da esquerda e SW12 o da direita)
set_property PACKAGE_PIN R2 [get_ports LD3]
    set_property IOSTANDARD LVCMOS33 [get_ports LD3]
set_property PACKAGE_PIN T1 [get_ports LD2]
    set_property IOSTANDARD LVCMOS33 [get_ports LD2]
set_property PACKAGE_PIN U1 [get_ports LD1]
    set_property IOSTANDARD LVCMOS33 [get_ports LD1]
set_property PACKAGE_PIN W2 [get_ports LD0]
    set_property IOSTANDARD LVCMOS33 [get_ports LD0]

# Carga do registo dos pontos decimais (interruptor SW11)
set_property PACKAGE_PIN R3 [get_ports LDDP]
    set_property IOSTANDARD LVCMOS33 [get_ports LDDP]

# Enable dos 4 displays (interruptor SW10)
set_property PACKAGE_PIN T2 [get_ports EN]
    set_property IOSTANDARD LVCMOS33 [get_ports EN]

# Clear de todos os registos (tecla BTNC)
set_property PACKAGE_PIN U18 [get_ports Clear]
    set_property IOSTANDARD LVCMOS33 [get_ports Clear]

# Segmentos a..g (Seg[6] = a ... Seg[0] = g), ativos a Low
set_property PACKAGE_PIN W7 [get_ports {Seg[6]}]
set_property PACKAGE_PIN W6 [get_ports {Seg[5]}]
set_property PACKAGE_PIN U8 [get_ports {Seg[4]}]
set_property PACKAGE_PIN V8 [get_ports {Seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {Seg[2]}]
set_property PACKAGE_PIN V5 [get_ports {Seg[1]}]
set_property PACKAGE_PIN U7 [get_ports {Seg[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Seg[*]}]

# Ponto decimal, ativo a Low
set_property PACKAGE_PIN V7 [get_ports DP]
    set_property IOSTANDARD LVCMOS33 [get_ports DP]

# Ânodos comuns dos 4 displays, ativos a Low (An[0] é o display da direita)
set_property PACKAGE_PIN U2 [get_ports {An[0]}]
set_property PACKAGE_PIN U4 [get_ports {An[1]}]
set_property PACKAGE_PIN V4 [get_ports {An[2]}]
set_property PACKAGE_PIN W4 [get_ports {An[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {An[*]}]

# Definir a tensão correta da placa Basys3
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

O *testbench* do circuito de teste (Listagem 24, no Anexo A) reproduz o ensaio na placa, também com `METADE = 1` (Figura 20): escreve "rdLE" com um impulso em cada `LDi`, acende os pontos dos *displays* 2 e 0 ("rd.LE."), mantém `LD0` ligado enquanto `Data` passa por 14 e 15 (o *display* 0 acompanha-o e fica com o "-"), desliga e volta a ligar `EN` (os valores mantêm-se) e termina com `Clear` ("0000"). Na Figura 21, com `Pontos` = 0101, `DP` vai a 0 só nos *displays* 2 e 0.

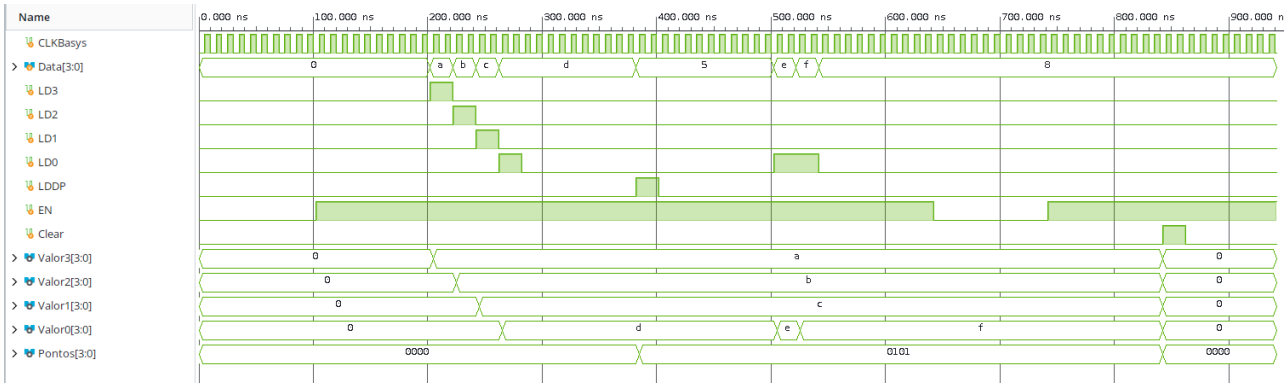


Figura 20: Simulação do circuito de teste `TesteDisplays` no Vivado (METADE = 1).

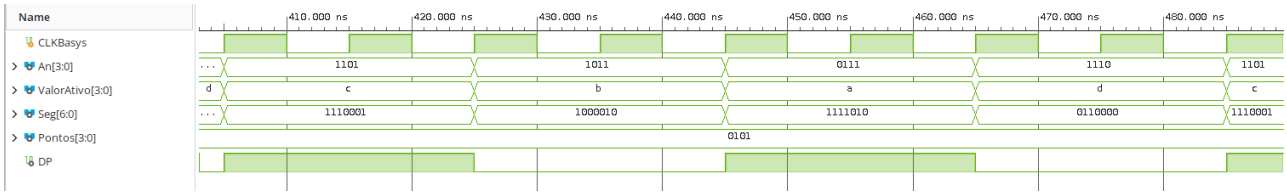
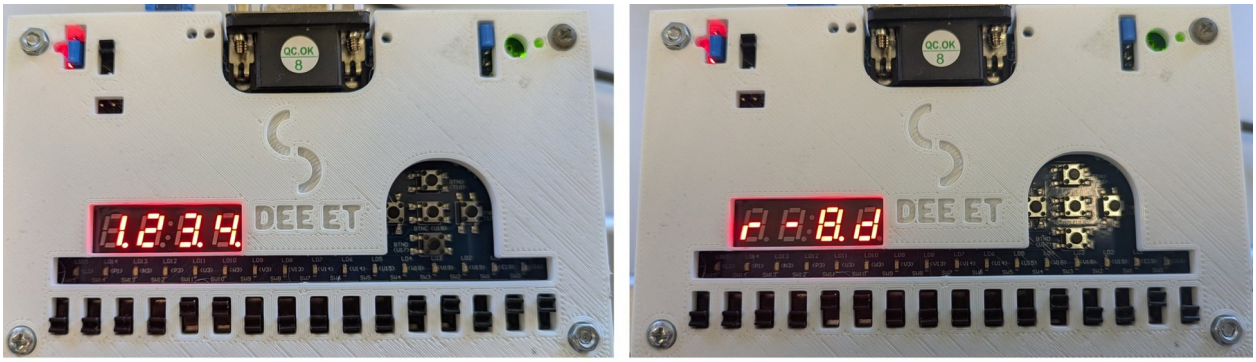


Figura 21: Pormenor da simulação do circuito de teste: um varrimento completo com "rd.LE.".

O *bitstream* foi gerado sem erros (20 LUT e 55 *flip-flops*). Ensaio na placa:

1. ligar SW10 (EN): "0000";
2. carregar 1, 2, 3 e 4 com SW3 a SW0 e os interruptores SW15 a SW12: "1234";
3. carregar os pontos com 1011 (SW11): "1.23.4." (Figura 22 (a));
4. carregar 10, 15, 8 e 11 e os pontos com 0010: "r-8.d" (Figura 22 (b));
5. premir BTNC: "0000", sem pontos (Figura 23).



(a) **(b)**

Figura 22: Ensaio do controlador na placa Basys3: (a) "1.23.4."; (b) "r-8.d", com símbolos originais.

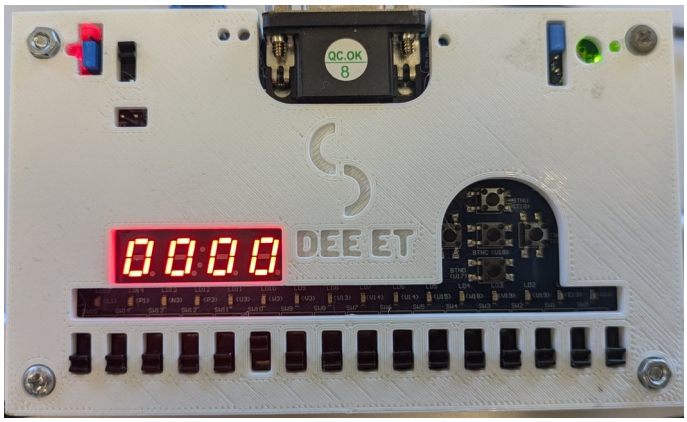


Figura 23: Ensaio do controlador na placa Basys3 depois do Clear (BTNC): "0000", sem pontos.

9 Análise crítica do código de fontes externas

Analisa-se aqui o código que não foi escrito de raiz: o gerado pelo Digital, os exemplos do Moodle (com erros intencionais), a primitiva FDCE e o apoio de IA generativa.

9.1 Código gerado pelo Digital

Os ficheiros gerados pelo Digital foram usados sem alterações: são corretos e legíveis, com uma atribuição contínua por saída, e os *testbenches* confirmaram que a exportação não alterou a função dos circuitos.

9.2 Transcodificador do Moodle

O `Transcod7Seg` do Moodle tem a estrutura certa, mas precisou das correções da Tabela 8.

Tabela 8: Erros do transcodificador do Moodle e correções introduzidas.

Código do Moodle	Problema	Correção
seg <= ... e segmentos <= ...	Nomes não declarados (o Verilog distingue maiúsculas): não compila.	Atribuição sempre a Segmentos.
reg [7:0] Seg;	8 *bits* para 7 segmentos.	output reg [6:0] Segmentos, sem registo auxiliar.
4'b1001 : Seg <= 7'b0001100;	O 9 fica sem o segmento *d*.	7'b0000100.
Falta o caso 4'b1110	O 14 cai no default (apagado).	Um caso para cada valor.
4'b1011 : Seg <= 7'b1001111;	O 11 é igual ao 1.	Novos símbolos 10 a 15 ("rdLEC-").
always @(Numero) com <=	Num circuito combinatório é preferível always @(*) com =.	always @(*) com =.

Os símbolos 10 a 15 do Moodle (por exemplo "≡" e "°") foram substituídos pelos símbolos originais da secção 5.

9.3 Divisores de clock do Moodle

O `clk_div` do Moodle usa atribuições bloqueantes num bloco sequencial (má prática) e o comentário indica 1 kHz para 25 MHz (na Basys3 daria 4 kHz). O `Clock_Divider` tem:

- `HALFCYCLE` com o valor de teste 10 (saída de cerca de 4,5 MHz);
- contagem de 0 a `HALFCYCLE`, ou seja, meios períodos de `HALFCYCLE + 1` ciclos;
- indentação enganadora no último `else` e constantes de 25 *bits* num contador de 32.

O `DivisorClock` mantém a ideia, sem *enable* nem *reset*, com atribuições não bloqueantes e comparação com `METADE - 1`.

9.4 Contadores do Moodle

O "contador simples" do Moodle tem uma saída de 4 *bits* para 2 *bits* úteis, uma atribuição bloqueante e um segundo `always` desnecessário; o "contador em anel" não compila (declara `Qd` na lista de portas e `Q` no corpo) e não é preciso, porque o descodificador já faz essa função. O `Contador4` ficou reduzido a um registo de 2 *bits*.

9.5 Primitiva FDCE

A FDCE segue o modelo do Vivado; confirmou-se que `CLR` (assíncrono) e `CE` são ativos a *High*.

9.6 IA generativa

Foi usada IA generativa como apoio na escrita dos *testbenches*, na verificação do código Verilog, no desenho da Figura 12 (os 16 símbolos do transcodificador) e na redação deste relatório, bem como para confirmar que o relatório inclui todos os elementos pedidos no enunciado.

10 Conclusão

Percorreu-se o fluxo completo de projeto em FPGA, do Digital ao Vivado e à placa Basys3. Os módulos desenvolvidos (*multiplexers* 4x1 e 16x4, descodificador 2x4, transcodificador com os símbolos "rdLEC-", registo com FDCE, divisor de *clock* e contador de módulo 4) foram integrados no controlador dos quatro *displays*.

O trabalho consolidou a diferença entre descrição comportamental e estrutural, a reutilização de módulos por hierarquia e a importância dos *testbenches* (os 7 das secções 2 a 7 terminam com `Simulacao OK`), bem como o cuidado com a polaridade dos sinais da placa (ativos a *Low*) e com as restrições temporais de um relógio gerado pela lógica.

Todos os circuitos funcionaram como esperado na placa.

Anexo A Testbenches

Listagens dos *testbenches* das secções 3 a 8. Os das secções 3 a 7 verificam as saídas automaticamente e terminam com `Simulacao OK` ou `Simulacao FALHOU`; os dois da secção 8 servem para observar as formas de onda.

Listagem 17: Testbench do descodificador 2x4 (`descodificador2x4/descodificador2x4_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do descodificador 2x4 com enables individuais nas saídas
module descodificador2x4_teste1();

//   Declaração das entradas
     reg [1:0] B;          // Bit1, Bit0
     reg [3:0] E;          // En3..En0

//   Declaração das saídas
     wire [3:0] O;         // Out3..Out0

//   Instanciação da UUT e ligação das entradas e saídas
     descodificador2x4 UUT (
         .Bit1(B[1]),
         .Bit0(B[0]),
         .En0(E[0]),
         .En1(E[1]),
         .En2(E[2]),
         .En3(E[3]),
         .Out0(O[0]),
         .Out1(O[1]),
         .Out2(O[2]),
         .Out3(O[3])
     );

//   Declaração de vars auxiliares
     integer n;            // Contador para o ciclo for
     integer erros = 0;    // Número de saídas diferentes do esperado

//   Verificação automática: só a saída B pode estar ativa, e só se En[B]=1
     always @(B or E) begin
         #1;
         if (O !== ((4'b0001 << B) & E)) begin
             $display("ERRO t=%0t: B=%b E=%b O=%b (esperado %b)", $time, B, E, O, (4'b0001 << B) & E);
             erros = erros + 1;
         end
     end

//   Definição do valor das entradas ao longo do tempo
     initial begin
         B = 2'b00;        // Valor inicial das vars de entrada
         E = 4'b0000;      // Todas as saídas desativadas (O=0000)

         #200;             // Aguardar 200ns para simular o tempo
                           // das configurações iniciais da FPGA

         for (n=0; n<16; n=n+1)
         begin
             #50;                          // Passam 50ns a cada ciclo de n
             case (n)
                  0: E = 4'b1111;          // Ativa todos os enables (O=0001)
                  1: B = 2'b01;            // Descodifica 1 (O=0010)
                  2: B = 2'b10;            // Descodifica 2 (O=0100)
```

```
                3: B = 2'b11;            // Descodifica 3 (O=1000)
                4: E = 4'b0111;          // Desativa En3: saída 3 apaga (O=0000)
                5: B = 2'b10;            // Descodifica 2 (O=0100)
                6: E = 4'b0011;          // Desativa En2 (O=0000)
                7: B = 2'b01;            // Descodifica 1 (O=0010)
                8: E = 4'b0001;          // Desativa En1 (O=0000)
                9: B = 2'b00;            // Descodifica 0 (O=0001)
               10: E = 4'b1110;          // Só En0 desativado (O=0000)
               11: B = 2'b11;            // Descodifica 3 (O=1000)
               12: E = 4'b0100;          // Só En2 ativo, mas B=3 (O=0000)
               13: B = 2'b10;            // Descodifica 2 (O=0100)
               14: E = 4'b1011;          // Todos menos En2, B=2 (O=0000)
               15: E = 4'b0000;          // Desativa tudo (O=0000)
            endcase
        end
        #50;
        if (erros == 0) $display("Simulacao OK: descodificador2x4 sem erros");
        else            $display("Simulacao FALHOU: %0d erros", erros);
        $finish;
    end
endmodule
```

Listagem 18: Testbench do multiplexer 16x4 (`multiplexer16x4/multiplexer16x4_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do multiplexer 16x4
module multiplexer16x4_teste1();

//   Declaração das entradas
     reg [1:0] S;
     reg [3:0] D0, D1, D2, D3;

//   Declaração da saída
     wire [3:0] Z;

//   Instanciação da UUT e ligação das entradas e saídas
     multiplexer16x4 UUT (
         .DataIn0(D0),
         .DataIn1(D1),
         .DataIn2(D2),
         .DataIn3(D3),
         .DataOut(Z),
         .Sel(S)
     );

//   Declaração de vars auxiliares
     integer n;            // Contador para o ciclo for
     integer erros = 0;    // Número de saídas diferentes do esperado
     reg [3:0] esperado;

//   Verificação automática: a saída deve ser sempre igual à entrada selecionada
     always @(S or D0 or D1 or D2 or D3) begin
         #1;
         case (S)
             2'd0: esperado = D0;
             2'd1: esperado = D1;
             2'd2: esperado = D2;
             2'd3: esperado = D3;
         endcase
         if (Z !== esperado) begin
```

```
            $display("ERRO t=%0t: S=%d Z=%h (esperado %h)", $time, S, Z, esperado);
            erros = erros + 1;
        end
    end

//  Definição do valor das entradas ao longo do tempo
    initial begin
        S  = 2'b00;       // Valor inicial das vars de seleção
        D0 = 4'h0;        // Valor inicial das vars de entrada
        D1 = 4'h0;
        D2 = 4'h0;
        D3 = 4'h0;

        #200;             // Aguardar 200ns para simular o tempo
                          // das configurações iniciais da FPGA

        for (n=0; n<14; n=n+1)
        begin
            #50;                          // Passam 50ns a cada ciclo de n
            case (n)
                 0: begin                 // Valores distintos nas 4 entradas
                        D0 = 4'hA;        // (Z=A)
                        D1 = 4'h5;
                        D2 = 4'hC;
                        D3 = 4'h3;
                    end
                 1: S = 2'b01;            // Seleciona DataIn1 (Z=5)
                 2: S = 2'b10;            // Seleciona DataIn2 (Z=C)
                 3: S = 2'b11;            // Seleciona DataIn3 (Z=3)
                 4: D0 = 4'hF;            // Muda entrada não selecionada (Z=3)
                 5: D3 = 4'h9;            // Muda entrada selecionada (Z=9)
                 6: S = 2'b00;            // Seleciona DataIn0 (Z=F)
                 7: D0 = 4'h1;            // Testa cada bit isolado (Z=1)
                 8: D0 = 4'h2;            // (Z=2)
                 9: D0 = 4'h4;            // (Z=4)
                10: D0 = 4'h8;            // (Z=8)
                11: S = 2'b10;            // Seleciona DataIn2 (Z=C)
                12: D2 = 4'h0;            // (Z=0)
                13: S = 2'b01;            // Seleciona DataIn1 (Z=5)
            endcase
        end
        #50;
        if (erros == 0) $display("Simulacao OK: multiplexer16x4 sem erros");
        else            $display("Simulacao FALHOU: %0d erros", erros);
        $finish;
    end
endmodule
```

Listagem 19: Testbench do transcodificador (`transcod7seg/transcod7seg_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do transcodificador para 7 segmentos.
// Percorre os 16 valores de Numero e verifica, para cada um:
//   - que a saída é igual à tabela de referência (Fig. 30 e símbolos 10 a 15);
//   - que nenhum bit da saída fica indefinido (X ou Z);
//   - que os 16 símbolos são todos diferentes entre si (nenhum é ambíguo no display).
// Depois repete os 16 valores por ordem aleatória, para garantir que a saída
// só depende do valor atual e não do valor anterior (circuito combinatório).
module transcod7seg_teste1();

```

```
12  //   Declaração das entradas
13       reg [3:0] N;
14
15  //   Declaração da saída (a..g, ativos a Low)
16       wire [6:0] S;
17
18  //   Instanciação da UUT e ligação das entradas e saídas
19       Transcod7Seg UUT (
20           .Numero(N),
21           .Segmentos(S)
22       );
23
24  //   Tabela de referência, escrita com os segmentos ACESOS (ativo a High) para
25  //   ser fácil de comparar com a Fig. 30; a saída esperada é a sua negação.
26       reg [6:0] acesos [0:15];
27       initial begin           //  abcdefg
28           acesos[0]  = 7'b1111110;  acesos[1]  = 7'b0110000;
29           acesos[2]  = 7'b1101101;  acesos[3]  = 7'b1111001;
30           acesos[4]  = 7'b0110011;  acesos[5]  = 7'b1011011;
31           acesos[6]  = 7'b1011111;  acesos[7]  = 7'b1110000;
32           acesos[8]  = 7'b1111111;  acesos[9]  = 7'b1111011;
33           acesos[10] = 7'b0000101;  acesos[11] = 7'b0111101;  // r  d
34           acesos[12] = 7'b0001110;  acesos[13] = 7'b1001111;  // L  E
35           acesos[14] = 7'b1001110;  acesos[15] = 7'b0000001;  // C  -
36       end
37
38  //   Declaração de vars auxiliares
39       integer n, m, k, erros = 0, verificacoes = 0;
40       integer semente = 7;
41       reg [6:0] obtido [0:15];
42       reg [3:0] ordem [0:15];
43       reg [3:0] tmp;
44
45       task verifica(input [3:0] valor);
46           begin
47               verificacoes = verificacoes + 1;
48               if (^S === 1'bx) begin
49                   $display("ERRO: Numero=%0d tem segmentos indefinidos (%b)", valor, S);
50                   erros = erros + 1;
51               end
52               else if (S !== ~acesos[valor]) begin
53                   $display("ERRO: Numero=%0d Segmentos=%b (esperado %b)", valor, S, ~acesos[valor]);
54                   erros = erros + 1;
55               end
56           end
57       endtask
58
59  //   Definição do valor das entradas ao longo do tempo
60       initial begin
61           N = 4'h0;
62           #200;                    // Aguardar as configurações iniciais da FPGA
63
64           // 1) Os 16 valores por ordem crescente
65           for (n=0; n<16; n=n+1)
66           begin
67               N = n;
68               #50;                 // Passam 50ns em cada valor
69               verifica(n);
70               obtido[n] = S;
71           end
72
73           // 2) Os 16 símbolos têm de ser todos diferentes
```

```
        for (n=0; n<16; n=n+1)
            for (m=n+1; m<16; m=m+1)
                if (obtido[n] === obtido[m]) begin
                    $display("ERRO: Numero=%0d e Numero=%0d mostram o mesmo simbolo", n, m);
                    erros = erros + 1;
                end

        // 3) Os 16 valores por ordem aleatória (baralhados), duas vezes
        for (k=0; k<2; k=k+1) begin
            for (n=0; n<16; n=n+1) ordem[n] = n;
            for (n=15; n>0; n=n-1) begin
                m = {$random(semente)} % (n+1);
                tmp = ordem[n]; ordem[n] = ordem[m]; ordem[m] = tmp;
            end
            for (n=0; n<16; n=n+1) begin
                N = ordem[n];
                #20 verifica(ordem[n]);
            end
        end

        if (erros == 0) $display("Simulacao OK: Transcod7Seg sem erros (%0d verificacoes)", verificacoes);
        else            $display("Simulacao FALHOU: Transcod7Seg com %0d erros", erros);
        $finish;
    end
endmodule
```

Listagem 20: Testbench do registo de 4 bits (`registo4/registo4_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do registo de 4 bits com clear e enable.
// Parte 1: casos dirigidos (CE, clear assíncrono, prioridade do clear, 16 valores).
// Parte 2: 2000 ciclos com entradas aleatórias comparados com um modelo de referência,
//          com CLR a mudar em instantes que não coincidem com o relógio.
module registo4_teste1();

//   Declaração das entradas
     reg [3:0] D = 4'h0;
     reg CE = 0, CLR = 0;
     reg Clk = 0;

//   Declaração da saída
     wire [3:0] Q;

//   Instanciação da UUT e ligação das entradas e saídas
     Registo4 UUT (
         .Data(D),
         .CE(CE),
         .Clk(Clk),
         .CLR(CLR),
         .DataOut(Q)
     );

//   Relógio de 100 MHz (período de 10ns), como o da Basys3
     always #5 Clk = ~Clk;

//   Modelo de referência: o comportamento esperado do registo
     reg [3:0] referencia = 4'h0;
     always @(posedge Clk or posedge CLR)
         if (CLR)      referencia <= 4'h0;
         else if (CE)  referencia <= D;
```

```
34
35  //   Verificação contínua: a saída tem de ser sempre igual à referência
36  //   (comparada 1ns depois de qualquer mudança, para dar tempo à UUT)
37       integer erros = 0, verificacoes = 0;
38       always @(Q or referencia) begin
39           #1;
40           verificacoes = verificacoes + 1;
41           if (Q !== referencia) begin
42               if (erros < 10) $display("ERRO t=%0t: Q=%h (esperado %h) D=%h CE=%b CLR=%b", $time, Q,
    referencia, D, CE, CLR);
43               erros = erros + 1;
44           end
45       end
46
47  //   Verificação direta num instante (para os casos dirigidos)
48       task verifica(input [3:0] esperado, input [8*40:1] caso);
49           begin
50               verificacoes = verificacoes + 1;
51               if (Q !== esperado) begin
52                   $display("ERRO t=%0t (%0s): Q=%h (esperado %h)", $time, caso, Q, esperado);
53                   erros = erros + 1;
54               end
55           end
56       endtask
57
58  //   A saída só pode mudar num flanco ascendente do relógio ou com CLR ativo
59       time ultimo_flanco = -1;
60       always @(posedge Clk) ultimo_flanco = $time;
61       always @(Q)
62           if ($time > 0 && $time != ultimo_flanco && CLR !== 1'b1) begin
63               $display("ERRO t=%0t: Q mudou para %h fora do flanco do relogio e sem CLR", $time, Q);
64               erros = erros + 1;
65           end
66
67       integer n, v;
68       integer semente = 2026;        // semente fixa: resultado repetível
69       initial begin
70           // --- Parte 1: casos dirigidos ------------------------------------------
71           #1   verifica(4'h0, "valor inicial apos configuracao (INIT=0)");
72           #100;                                          // configurações iniciais da FPGA
73           // As entradas mudam a meio do período (relógio sobe em 5, 15, 25, ...)
74           D = 4'h5; CE = 0;  #10 verifica(4'h0, "CE=0 nao carrega");
75           CE = 1;            #10 verifica(4'h5, "CE=1 carrega no flanco");
76           D = 4'hA;          #3  verifica(4'h5, "entre flancos a saida mantem-se");
77                              #7  verifica(4'hA, "carrega A");
78           CE = 0; D = 4'h3;  #50 verifica(4'hA, "CE=0 mantem durante 5 ciclos");
79           #2 CLR = 1;        #1  verifica(4'h0, "CLR assincrono (sem flanco)");
80           CE = 1; D = 4'hF;  #20 verifica(4'h0, "CLR tem prioridade sobre CE");
81           CLR = 0;           #0.5 verifica(4'h0, "apos CLR so carrega no flanco seguinte");
82                              #6.5 verifica(4'hF, "carrega F apos libertar CLR");
83           // Os 16 valores possíveis, carregados um a um (cada bit em 0 e em 1)
84           for (v = 0; v < 16; v = v + 1) begin
85               D = v; #10 verifica(v, "carregamento dos 16 valores");
86           end
87           // Bits independentes: 1 a andar e 0 a andar
88           for (v = 0; v < 4; v = v + 1) begin
89               D = 4'b0001 << v;     #10 verifica(4'b0001 << v, "um bit a 1");
90               D = ~(4'b0001 << v);  #10 verifica(~(4'b0001 << v), "um bit a 0");
91           end
92
93           // --- Parte 2: entradas aleatórias contra o modelo de referência ----------
94           for (n = 0; n < 2000; n = n + 1) begin
```

```
            // instante aleatório entre 0 e 3ns depois do flanco descendente,
            // nunca em cima do flanco ascendente (evita corridas no testbench)
            @(negedge Clk); verifica(referencia, "ciclo aleatorio");
            #({$random(semente)} % 4);
            D   = $random(semente);
            CE  = $random(semente);
            CLR = ({$random(semente)} % 8 == 0);                 // clear em ~1/8 das vezes
        end
        #2 CLR = 0; #20;                                 // sem impulsos de largura nula no CLR

        if (erros == 0) $display("Simulacao OK: Registo4 sem erros (%0d verificacoes)", verificacoes);
        else            $display("Simulacao FALHOU: Registo4 com %0d erros", erros);
        $finish;
    end
endmodule
```

Listagem 21: Testbench do divisor de clock (`divisorclock/divisorclock_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do divisor de clock.
// Verifica, para vários valores do parâmetro METADE, que ClockOut:
//   - tem período 2 x METADE ciclos de ClockIn (f_out = f_in / (2 x METADE));
//   - tem duty cycle de 50% (METADE ciclos a 1 e METADE ciclos a 0);
//   - só muda no flanco ascendente de ClockIn;
//   - mantém esse comportamento ao longo de muitos períodos.
// Inclui o valor usado na placa (METADE = 25000), simulando 12 períodos de saída (6 ms),
// e confirma que a frequência real fica entre 1 kHz e 10 kHz.
module divisorclock_teste1();

//   Declaração das entradas: relógio de 100 MHz (período de 10ns), como o da Basys3
     reg Clk = 0;
     always #5 Clk = ~Clk;

//   Declaração das saídas
     wire Out1, Out2, Out5, Out7, OutPlaca;

//   Instanciação das UUT com vários valores de METADE (o circuito é o mesmo)
     DivisorClock #(.METADE(1))  UUT1 (.ClockIn(Clk), .ClockOut(Out1));
     DivisorClock #(.METADE(2))  UUT2 (.ClockIn(Clk), .ClockOut(Out2));
     DivisorClock #(.METADE(5))  UUT5 (.ClockIn(Clk), .ClockOut(Out5));
     DivisorClock #(.METADE(7))  UUT7 (.ClockIn(Clk), .ClockOut(Out7));
     DivisorClock                UUTP (.ClockIn(Clk), .ClockOut(OutPlaca));   // valor por omissão

//   Instante do último flanco ascendente de ClockIn
     time flanco_in = 0;
     always @(posedge Clk) flanco_in = $time;

     integer erros = 0;

//   Monitor genérico de uma saída: mede tempo a 1, tempo a 0 e período
     task automatic monitoriza(input integer metade, input integer periodos_min, input [8*12:1] nome);
         time t_sobe, t_desce, t_sobe_ant, alto, baixo;
         integer p;
         begin
             p = 0;
             t_sobe_ant = 0;
             while (p < periodos_min + 1) begin
                 case (metade)
                     1: @(posedge Out1);  2: @(posedge Out2);  5: @(posedge Out5);
                     7: @(posedge Out7);  default: @(posedge OutPlaca);
```

```
                endcase
                t_sobe = $time;
                if (t_sobe != flanco_in) begin
                    $display("ERRO %0s: ClockOut subiu fora do flanco de ClockIn (t=%0t)", nome, t_sobe);
                    erros = erros + 1;
                end
                if (p > 0) begin
                    alto  = t_desce - t_sobe_ant;
                    baixo = t_sobe - t_desce;
                    if (alto != metade*10 || baixo != metade*10) begin
                        $display("ERRO %0s: a 1 durante %0t ns e a 0 durante %0t ns (esperado %0d ns
cada)",
                                 nome, alto, baixo, metade*10);
                        erros = erros + 1;
                    end
                end
                case (metade)
                    1: @(negedge Out1);  2: @(negedge Out2);  5: @(negedge Out5);
                    7: @(negedge Out7);  default: @(negedge OutPlaca);
                endcase
                t_desce = $time;
                if (t_desce != flanco_in) begin
                    $display("ERRO %0s: ClockOut desceu fora do flanco de ClockIn (t=%0t)", nome, t_desce);
                    erros = erros + 1;
                end
                t_sobe_ant = t_sobe;
                p = p + 1;
            end
        end
    endtask

    reg fim1 = 0, fim2 = 0, fim5 = 0, fim7 = 0, fimP = 0;
    initial begin monitoriza(1, 50, "METADE=1");  fim1 = 1; end
    initial begin monitoriza(2, 50, "METADE=2");  fim2 = 1; end
    initial begin monitoriza(5, 50, "METADE=5");  fim5 = 1; end
    initial begin monitoriza(7, 50, "METADE=7");  fim7 = 1; end
    initial begin monitoriza(25000, 12, "placa");  fimP = 1; end

    real freq;
    initial begin
        wait (fim1 && fim2 && fim5 && fim7 && fimP);
        freq = 100.0e6 / (2 * UUTP.METADE);
        $display("Frequencia de ClockOut na placa: %0.1f Hz", freq);
        if (freq < 1000.0 || freq > 10000.0) begin
            $display("ERRO: frequencia fora do intervalo 1 kHz a 10 kHz");
            erros = erros + 1;
        end
        if (erros == 0) $display("Simulacao OK: DivisorClock sem erros");
        else            $display("Simulacao FALHOU: DivisorClock com %0d erros", erros);
        $finish;
    end

//   Segurança: termina se alguma saída nunca chegar a mudar
    initial begin
        #10_000_000;
        $display("Simulacao FALHOU: DivisorClock nao produziu os periodos esperados (fim1=%b fim2=%b fim5=
%b fim7=%b placa=%b)",
                 fim1, fim2, fim5, fim7, fimP);
        $finish;
    end
endmodule
```

Listagem 22: Testbench do contador de módulo 4 (`contador4/contador4_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do contador de módulo 4.
// Verifica:
//   - o estado inicial (0 após a configuração da FPGA);
//   - a sequência 0, 1, 2, 3, 0, ... durante 100 flancos (25 voltas);
//   - que as saídas só mudam no flanco ascendente (nunca no descendente nem entre flancos);
//   - que muda exatamente uma vez por ciclo de relógio.
module contador4_teste1();

//   Declaração das entradas
     reg Clk = 0;

//   Declaração das saídas
     wire Q1, Q0;

//   Instanciação da UUT e ligação das entradas e saídas
     Contador4 UUT (
         .Clock(Clk),
         .Q1(Q1),
         .Q0(Q0)
     );

//   Relógio com período de 20ns
     always #10 Clk = ~Clk;

//   As saídas só podem mudar no instante de um flanco ascendente
     time flanco = -1;
     integer erros = 0, mudancas = 0;
     always @(posedge Clk) flanco = $time;
     always @({Q1, Q0})
         if ($time > 0) begin
             mudancas = mudancas + 1;
             if ($time != flanco) begin
                 $display("ERRO t=%0t: Q1Q0 mudou para %b%b fora do flanco ascendente", $time, Q1, Q0);
                 erros = erros + 1;
             end
         end

//   Verificação do estado depois de cada flanco ascendente
     reg [1:0] esperado = 2'b00;
     integer n;
     initial begin
         #1;
         if ({Q1, Q0} !== 2'b00) begin
             $display("ERRO: estado inicial %b%b (esperado 00)", Q1, Q0);
             erros = erros + 1;
         end
         for (n=0; n<100; n=n+1) begin
             @(posedge Clk); #1;
             esperado = esperado + 1;           // módulo 4: 3 + 1 = 0 em 2 bits
             if ({Q1, Q0} !== esperado) begin
                 $display("ERRO t=%0t: Q1Q0=%b%b (esperado %b)", $time, Q1, Q0, esperado);
                 erros = erros + 1;
             end
             @(negedge Clk); #1;                // a meio do ciclo continua igual
             if ({Q1, Q0} !== esperado) begin
                 $display("ERRO t=%0t: Q1Q0 mudou no flanco descendente", $time);
                 erros = erros + 1;
             end
         end
```

```
        // Em 100 ciclos: Q0 muda 100 vezes e Q1 50 vezes; o bloco always conta
        // eventos de mudança de {Q1,Q0}, que são 100 (um por ciclo)
        if (mudancas != 100) begin
            $display("ERRO: %0d mudancas em 100 ciclos (esperado 100)", mudancas);
            erros = erros + 1;
        end
        if (erros == 0) $display("Simulacao OK: Contador4 sem erros (100 ciclos)");
        else            $display("Simulacao FALHOU: Contador4 com %0d erros", erros);
        $finish;
    end
endmodule
```

Listagem 23: Testbench do controlador dos displays (`controlodisplays/controlodisplays_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do controlador dos displays (ControloDisplays).
// METADE = 1: cada display fica aceso durante 2 ciclos de relógio (20ns), para que o
// varrimento se veja numa simulação curta (na placa, com METADE = 25000, são 0,5 ms).
// Os comentários indicam o que deve ver-se nos displays, da esquerda (3) para a direita (0).
module controlodisplays_teste1();

//   Declaração das entradas
    reg [3:0] D3 = 0, D2 = 0, D1 = 0, D0 = 0, DotPoints = 0;
    reg [3:0] Load = 0, Clr = 0, En = 0;      // índice i = display i
    reg LoadDP = 0, ClrDP = 0;
    reg Clk = 0;

//   Declaração das saídas
    wire [6:0] Segmentos;
    wire DotPoint;
    wire [3:0] Anodos;

//   Instanciação da UUT e ligação das entradas e saídas
    ControloDisplays #(.METADE(1)) UUT (
        .Data3(D3), .Load3(Load[3]), .Clr3(Clr[3]), .Enable3(En[3]),
        .Data2(D2), .Load2(Load[2]), .Clr2(Clr[2]), .Enable2(En[2]),
        .Data1(D1), .Load1(Load[1]), .Clr1(Clr[1]), .Enable1(En[1]),
        .Data0(D0), .Load0(Load[0]), .Clr0(Clr[0]), .Enable0(En[0]),
        .DotPoints(DotPoints), .LoadDP(LoadDP), .ClrDP(ClrDP),
        .Clock(Clk),
        .Segmentos(Segmentos),
        .DotPoint(DotPoint),
        .Anodos(Anodos)
    );

//   Relógio de 100 MHz (período de 10ns), como o da Basys3
    always #5 Clk = ~Clk;

//   Definição do valor das entradas ao longo do tempo
//   (as entradas mudam 2ns depois de um flanco descendente; os registos carregam no ascendente)
    initial begin
        #102;                                   // displays desligados
        En = 4'b1111;                     #100;   // "0000"
        {D3, D2, D1, D0} = {4'd2, 4'd0, 4'd2, 4'd6};
        Load = 4'b1111;          #10 Load = 0; #90;   // carrega os 4 registos: "2026"
        DotPoints = 4'b0100;
        LoadDP = 1;            #10 LoadDP = 0; #90;   // ponto no display 2: "20.26"
        Load = 4'b0001;                         // Load0 mantido a 1: o registo 0 segue D0
        D0 = 4'd5; #20 D0 = 4'd6; #20 D0 = 4'd7; #20;
        Load = 4'b0000; D0 = 4'd1;        #100;   // Load0 a 0: fica o 7: "20.27"
```

```
48          {D3, D2, D1, D0} = {4'd10, 4'd11, 4'd12, 4'd13};
49          Load = 4'b1010;          #10 Load = 0; #90;   // só os registos 3 e 1: "r0.L7"
50          En = 4'b1011;                          #100;   // display 2 desligado: "r L7"
51          #6 Clr[3] = 1; #2 Clr[3] = 0;         #92;   // clear assíncrono (entre flancos) do registo 3: "0
    L7"
52          En = 4'b1111; ClrDP = 1; #10 ClrDP = 0; #90;  // display 2 ligado, sem pontos: "00L7"
53          Clr = 4'b1111;            #10 Clr = 0; #90;   // todos os registos a 0: "0000"
54          En = 4'b0000;                          #100;   // displays desligados
55          $finish;
56      end
57
58  endmodule
```

Listagem 24: Testbench do circuito de teste (`controlodisplays/testedisplays_teste1.v`)

```
1   `timescale 1ns / 1ps
2
3   // Testbench do circuito de teste na placa (TesteDisplays, Fig. 32).
4   // Reproduz o que se faz na Basys3: escolher um valor nos interruptores Data e ligar e
5   // desligar o interruptor LDx do display pretendido.
6   // METADE = 1: cada display fica aceso durante 2 ciclos de relógio (20ns); na placa são 0,5 ms.
7   // Os comentários indicam o que deve ver-se nos displays, da esquerda (3) para a direita (0).
8   module testedisplays_teste1();
9
10  //   Declaração das entradas
11       reg [3:0] Data = 0;
12       reg LD3 = 0, LD2 = 0, LD1 = 0, LD0 = 0, LDDP = 0;
13       reg EN = 0, Clear = 0;
14       reg CLKBasys = 0;
15
16  //   Declaração das saídas
17       wire [6:0] Seg;
18       wire DP;
19       wire [3:0] An;
20
21  //   Instanciação da UUT e ligação das entradas e saídas
22       TesteDisplays #(.METADE(1)) UUT (
23           .Data(Data),
24           .LD3(LD3), .LD2(LD2), .LD1(LD1), .LD0(LD0), .LDDP(LDDP),
25           .EN(EN),
26           .Clear(Clear),
27           .CLKBasys(CLKBasys),
28           .Seg(Seg),
29           .DP(DP),
30           .An(An)
31       );
32
33  //   Relógio de 100 MHz da Basys3
34       always #5 CLKBasys = ~CLKBasys;
35
36  //   Definição do valor das entradas ao longo do tempo
37  //   (as entradas mudam 2ns depois de um flanco descendente)
38       initial begin
39           #102;                                  // displays desligados
40           EN = 1;                          #100;   // "0000"
41           Data = 4'd10; LD3 = 1; #20 LD3 = 0;    // r no display da esquerda
42           Data = 4'd11; LD2 = 1; #20 LD2 = 0;    // d
43           Data = 4'd12; LD1 = 1; #20 LD1 = 0;    // L
44           Data = 4'd13; LD0 = 1; #20 LD0 = 0;    // E
45                                          #100;   // "rdLE"
46           Data = 4'b0101; LDDP = 1; #20 LDDP = 0;
```

```
47                                      #100;   // pontos nos displays 2 e 0: "rd.LE."
48          LD0 = 1;                            // LD0 ligado: o display 0 segue Data
49          Data = 4'd14; #20 Data = 4'd15; #20;
50          LD0 = 0; Data = 4'd8;           #100;   // LD0 desligado: fica o 15 e Data já não conta: "rd.L-."
51          EN = 0;                         #100;   // displays desligados
52          EN = 1;                         #100;   // voltam a mostrar o mesmo: "rd.L-."
53          Clear = 1;       #20 Clear = 0; #80;   // tudo a 0 e pontos apagados: "0000"
54          $finish;
55      end
56
57  endmodule
```